Botany Downs Secondary College
Internal Assessment

Level 3-2026

91906- Use Complex Techniques to Develop a Computer Program-6 credits

91907- Use complex processes to develop a digital technologies outcome -6 credits

Achieved	Merit	Excellence
• Use complex programming techniques to develop a computer program.	• Use complex programming techniques to develop an informed computer program.	• Use complex programming techniques to develop a refined computer program.

Student Declaration:

I I Krish Ram hereby declare that I have completed the assessment for 91906-907, independently and to the best of my abilities. This assessment represents my own work and is based on my own research, practice and understanding of the subject matter.

I confirm that all sources used in this assessment, including but not limited to books, articles, online resources, and any other references, have been appropriately cited and acknowledged according to the prescribed referencing style.

I further affirm that I have not engaged in any form of academic dishonesty, such as plagiarism or unauthorized collaboration, in the completion of this assessment. The ideas, arguments and content presented in this assessment are my own and have not been copied or reproduced from any other source.

I understand that any act of academic misconduct or violation of the academic integrity may result in disciplinary actions, which could include penalties such as grade reduction, course failure or other consequences as determined by the institution.

I take full responsibility for the authenticity and originality of the assessment and acknowledge that my work will be subject to scrutiny and evaluation by my instructors or assessors.

Signed : Krish Ram Date: 28/08/26

Emergency Q/WaitSmart NZ

1. Purpose of the digital outcome

Describe the purpose of your digital outcome.

My program is called Emergency Q. It is a desktop app that shows how busy Auckland's Emergency Departments are, so a person can decide where to go for urgent care that is not life threatening. It lists the hospitals, shows each one's estimated wait time with a green, orange or red colour so you can read it quickly, and lets you type your suburb to sort the list from closest to furthest. This is a prototype. It takes the main idea from my inquiry, that ED wait times are basically hidden from the public, and turns it into something that actually runs.

Who is the intended user?

The main users are ordinary people who have an urgent but not life threatening health problem, like a deep cut, a high fever or a possible broken bone, and are trying to work out which ED to go to. My stakeholder research backed this up. Out of the nine people I surveyed, almost everyone said they would use an app that showed live wait times, and the two people who were most frustrated were the ones who had been caught out by long waits the most often. The users are not all the same though. Alan Taylor, who was 65 or older, only wanted the wait time and nothing else, while Benjamin Lau wanted patient counts and more detail. So the design has to be simple enough for a stressed or older person but still useful to someone who wants more. Carers and family members checking on behalf of someone else are a second group of users.

What problem does it solve?

At the moment there is no single public place in New Zealand where you can compare how busy different EDs are. Hospitals have the data but they do not share it in a way the public can use. Because of that, most people just go to their nearest ED and hope for the best, and sometimes get there to find a four to six hour wait when a hospital nearby was quieter. Emergency Q deals with the everyday side of that problem. It puts the hospitals on one screen, shows each one's wait and how many people are waiting, colours it so it is easy to read, and sorts by distance from your suburb. The comparison people cannot make right now becomes one quick step.

Why is this problem worth solving?

It is worth solving because it affects a lot of people and it has real consequences. One of my research sources (the NZ Herald, using official OIA data) found that Wellington Hospital alone had 575 code red events and about 3,200 patients who left without being treated in a single year. When people cannot tell how long they will wait, some EDs get overloaded while others sit quieter, patients get frustrated, and a few leave before they are seen, which can make their health worse. Even a simple, honest view of wait times can help spread people out and cut down on wasted trips. The idea is also proven. Queensland Health already runs a public ED dashboard in a health system a lot like ours, so I am not just guessing that it can work.

2. Programming language, software and development tools

Why Python and Tkinter?

I am writing the program in Python with the Tkinter toolkit. Python suits this project because it is easy to read, which matters when the code has to go through multiple versions and I need to change things quickly. It also lets me spend my time on the actual logic, like sorting hospitals by distance and matching a search, instead of fighting the language. Tkinter comes built into Python, so there is nothing extra to install and the prototype runs on any school or home computer that already has Python. It gives me the widgets I need, which are a window, a search box, a list, labels and buttons, and it lets me set colours directly. I need that for the green, orange and red wait system that my stakeholders said was the most important feature. Tkinter is event driven, so it fits an app where the user types, clicks a hospital and watches the details change.

Why VS Code?

I am using Visual Studio Code to write the code. It has good Python support built in, so I get syntax highlighting, autocomplete and error underlines that catch mistakes before I even run the program. It also has a terminal built in, so I can run and test each version without leaving the editor, and a linting extension that checks my code follows Python's naming and layout rules. It also connects to Git and GitHub, so I can commit and push straight from the editor.

Why GitHub?

I am using GitHub with Git to save my work and keep track of versions. This assessment wants clear evidence of managing different versions, and Git keeps the full history of every change with a date and a message. If a new version breaks something, I can look back at, or go back to, an older version that worked instead of losing everything. GitHub also stores the whole thing online, so I have a backup that is not just on my laptop, and a link I can send to my teacher and stakeholders when I want feedback. Keeping my commits tidy, one for each real change, also gives an honest record of how the program grew from each version.

3. Introduction to the chosen task

Background information

This task carries on from my Level 3 inquiry, where I looked into public access to real time health information in New Zealand. My final inquiry question was about the technical, ethical and regulatory challenges of building a real time dashboard for ED wait times, and how you could deal with them to make something feasible, trustworthy and open to the public. That inquiry ended in a proposal for a tool called WaitSmart NZ. Emergency Q is the programming version of that proposal, cut down to something I can actually build, test and improve inside this standard.

Objectives

- Build a working Tkinter app that lists Auckland EDs with their estimated wait times.

- Let the user type a suburb and re-sort the hospitals from closest to furthest using their coordinates.
- Show each hospital's wait as a green, orange or red status that is quick to read.
- Read the hospital and suburb data from a file instead of hard coding it, so it is easy to update.
- Show at least two complex techniques (a GUI, file handling, classes and objects, a custom type, and a third party library).
- Make the final program tidy, flexible and reliable, and test it properly across the multiple versions.

Significance

What makes this worth doing is that it takes a real, well documented problem and turns it into something you can actually try. It is not just an exercise, because every design choice comes back to what my stakeholders said. The colour coding is there because people asked for a signal they could read quickly. The distance sort is there because people default to the nearest ED with no way to compare. Building it also lets me practise the harder parts of programming, like GUIs, file handling, classes and a queue, on a project that means something to me.

4. Methodology

I am using an iterative, Agile style approach, planned on a Trello board, and building the program in multiple versions. Each version adds something or improves something, gets tested, and then gets looked at by my teacher and two other people before I start the next one. This lets me break the problem into small pieces, try different ways of building each piece, and feed what I learn from testing and feedback back into the design.

The main techniques I will use are:

- A distance calculation using the Haversine formula, which turns two latitude and longitude points into a real distance in kilometres so I can rank hospitals by how far they are from the user.
- A suburb dropdown whose selected name is looked up directly in a dictionary of coordinates, so the chosen suburb is always valid and found instantly.
- threshold logic that turns a wait time into a colour (green under 30 minutes, orange under 90, red above that), using named constants so the numbers are easy to change.
- file reading that loads the hospital and suburb records from a text file, skipping comments and blank lines, so I can add data without touching the code.
- an object oriented structure where one App class holds the interface and each hospital is its own object, keeping the interface and the logic apart.

The tools behind all of this are VS Code for writing and running code, Git and GitHub for versions and backups, Trello for managing tasks, Figma for wireframes and Figma for flowcharts.

5. Software requirements

Requirement	Detail
Programming language	Python 3.12. Any Python 3.10 or newer will run it.
Core library	Tkinter, which comes with a normal Python install, so there is nothing extra to add.
Standard library modules	math for the distance calculation, random to simulate live wait times in the prototype, os to find the data file, and threading and webbrowser in the later map versions.
Possible other libraries (later versions)	tkintermapview and requests, used from the map version onward. Version 1 uses only the standard library, so it has no outside dependencies.
Data files	A plain text or CSV file for the suburb coordinates and the hospital records, read when the program starts.
Editor and tools	Visual Studio Code with the Python extension, and Git for version control.
Operating system	Windows 10 or 11, macOS or Linux. Tkinter runs on all of them.
Hardware	Any normal laptop or desktop that can run Python 3. Version 1 needs no special hardware or internet.

6. List of complex techniques being used

The standard asks for at least two complex techniques. I plan to use several, and bring it in bit by bit across the versions.

Complex technique	How it will be used in Emergency Q
Graphical user interface (GUI)	A full Tkinter interface with a window, search box, scrollable list, detail panel and buttons
Reading and writing files	The hospital and suburb data is read from an outside text or CSV file when the program starts, so none of it is hard coded.
Classes and objects	One App class (which holds a tk.Tk window) holds the interface, and each hospital is made into an object, which keeps the interface separate from the logic.
A custom type	A Hospital type (a class) models each hospital's name, coordinates, phone, type and current wait, instead of loose separate variables.
A complex data structure	A queue, which fits the name Emergency Q, keeps track of recently viewed hospitals, showing a structure that is more than a plain list.
Third party library (later)	tkintermapview adds an interactive map of the hospitals in a later version.

7. List of complex processes being used

Complex process	How it will be applied
Project management with real tools	Trello board (To Do, Doing, Finished), plus GitHub for versions and Figma for design.
Breaking the problem into pieces	The app is split into small parts (the data, the search, the distance sort, the colour coding, the list and the detail view) and each one is built and tested on its own before being joined up.
Trying different components	I try more than one way of doing the same job, for example a plain Listbox against custom card frames, or a typed search against a dropdown, and pick the better one with a reason.
Versioning	Multiple versions, each one adding or improving something, with tidy file names, commits and online backups.

Managing feedback	After every version I get feedback from my teacher and two other people, then write up what I plan to change next.
Testing	Each part is tested with normal, boundary and invalid data, written up in the testing tables, and used to drive fixes.

8. Planning requirements

I am planning the design with a few different tools, each for a different part of it.

- Figma for the wireframes. I sketch each version's screen layout and label it before I write any code, so I know what I am building. The Version 1 wireframe goes with this document.
- Figma for the flowcharts. I draw the logic for each version (starting up, searching, sorting by distance, colour coding and showing details) so I can check the flow before I code it.
- Trello for the tasks. I break the whole project into cards and move them through To Do, Doing and Finished so I can see where I am.
- GitHub for saving and versioning, so every version is backed up and I can go back if I need to.

9. Relevant implications

These are the implications that actually matter for building this program. How I ended up applying each one is written up at the end of the portfolio.

Relevant implication	Describe	Explain (why it matters here)
Functionality	The program has to reliably show and sort the wait information, and be clear about what a "wait time" actually means.	If the data is old or badly labelled the app could send someone to the wrong hospital, so this is partly an ethical issue, not just a technical one. Wait times will be labelled clearly and there will be a note to call 111 in a real emergency.
Usability and accessibility	The screen has to be easy to read and use, including for older or vision impaired people, and under stress.	This pushes me towards clear colour contrast, larger readable text, decent sized click targets, and a colour status that does not depend on reading numbers.
Aesthetics	The look has to feel calm and clinical, because a stressful design makes an already stressful situation worse.	I kept one restrained palette defined as named constants at the top of the program, so every part of the screen uses the same white background, the same dark ink colour and the same three status colours, and nothing drifts out of step. Colour is only ever used to carry meaning, never as decoration, and the one loud element on the screen is the red Call 111 button, which is the thing that should stand out. Generous spacing and a single font at three sizes keep the list scannable instead of crowded. In an emergency context the appearance is doing a functional job, not just a decorative one.
Legal and copyright	Everything I use, meaning the data, libraries and images, has to be legal to use and credited.	I am only using open source libraries like Tkinter and tkintermapview within their licences, public hospital contact details, and images I either made myself or am allowed to use, with credits.
Privacy and data protection	The idea is built on health data, which is sensitive under the Privacy Act 2020 and the Health Information Privacy Code 2020.	The program only ever handles totals, meaning an estimated wait and a count of patients waiting, and never anything about an individual patient, so no health information about a person is stored or shown. There is no account, no login, and no personal data collected from the user at all. The suburb the user picks is held in memory for that session only and is never written to a file or sent anywhere, so nothing about their location survives closing the program. This matters for later versions too: if I add real GPS location or a feedback form, that becomes personal information and would need consent and a clear purpose before I collect it.
Future changes	The data, meaning the hospitals, suburbs and thresholds, will need updating over time.	Reading data from files and using named constants means I can update it without rewriting code, which keeps the program flexible.

Commented [LN1]: Please write these in this particular order.
Functionality
Usability
Accessibility
Aesthetics
Copyrights
Privacy and Data Protection
Future Changes

Equity	An app that only works on a phone or computer can leave out people without devices or internet.	Keeping it light, dependency free early on, and simple enough for a family member to check for someone else lowers that barrier. The wider idea is also meant to run as a website.

Links to Project Management:

Project Management Tool	Link
Trello	Version 1: https://trello.com/invite/b/6a5c3bfefc9df34bab75deef/ATTI4776f13fd4a8dcfe79fc6aef9cdb0f48F7AF035E/3dip-emergency-q-version-1 Version 2: https://trello.com/invite/b/6a86f7dab62889387c3e4721/ATTI5034932a7030b662f319b2a075d7a30a8DF57C7C/3dip-emergency-q-version-2 Version 3: https://trello.com/invite/b/6a8e1448c0ed48b348e266bf/ATTI22bd10a99a7d26c4220d062bf1b72867CD371E52/3dip-emergency-q-version-3
Github	https://github.com/RAMpage02496/Waitsmart-nz-internal.git

10. Iteration 1: planned actions

What Version 1 will do

Version 1 is the base version. The point of it is to get the core interaction working before I add distance sorting, live updates or a map. In this version the program will:

- open one window titled "Emergency Q - Version 1".
- hold a starter set of Auckland hospitals, each with a name, wait time, phone number and type (Public or Private).
- show all the hospitals in a scrollable list.
- have a search box that filters the list as you type, ignoring capital letters.
- let you click a hospital and see its full details (name, type, wait, phone) in a panel below the list.

This covers the three control structures the standard asks for. Sequence is the start up that builds the window and loads the data. Selection is the IF that decides whether a hospital matches the search and which details to show. Iteration is the FOR loop that goes through the hospital list to fill the display.

Planned code structure

Even though Version 1 is small, I want to lay it out so the later versions can grow from it without a rewrite. The plan keeps the data, the logic and the interface in separate parts:

- a data area at the top for the HOSPITALS data, and later the colour palette and wait thresholds as named constants.
- logic functions that do the thinking, like filtering the list and finding the selected hospital, kept away from the interface
- interface code that builds the window, list, search box and detail panel and connects them to the logic through event bindings.

From Version 2 on, this turns into a proper App class that holds the tk.Tk window, so the interface and its state live in one object and the GUI stays separate from the logic underneath.

Planned classes (UML)

This is the object design I am aiming for across the whole project. Version 1 mainly uses the App container and the Hospital record.

Class	Key attributes	What it does
App (holds a tk.Tk window)	hospitals, user_lat, user_lng, the widgets (search box, list, detail labels)	Sets up and runs the interface: __init__, build_ui(), update_list(), on_select(), on_search().
Hospital (custom type)	name, type, wait, phone, lat, lng	Holds one Emergency Department as a single object, and gives its display text and (later) its distance from the user.

Planned functions (Version 1)

Function	Purpose
update_list(search_term="")	Clears the list and rebuilds it, only adding hospitals whose name contains the search text (the loop plus the IF).
on_select(event)	Reads the hospital that was clicked and puts its name, type, wait and phone into the detail labels.
on_search(event)	Runs on each key press in the search box and calls update_list with the current text.

Key sections

The two most important parts of Version 1 are the live search filter and the split between the data and the display. The search filter matters because it is the first real bit of logic, and the distance sort in Version 2 will be built on the same pattern. The split between data and display

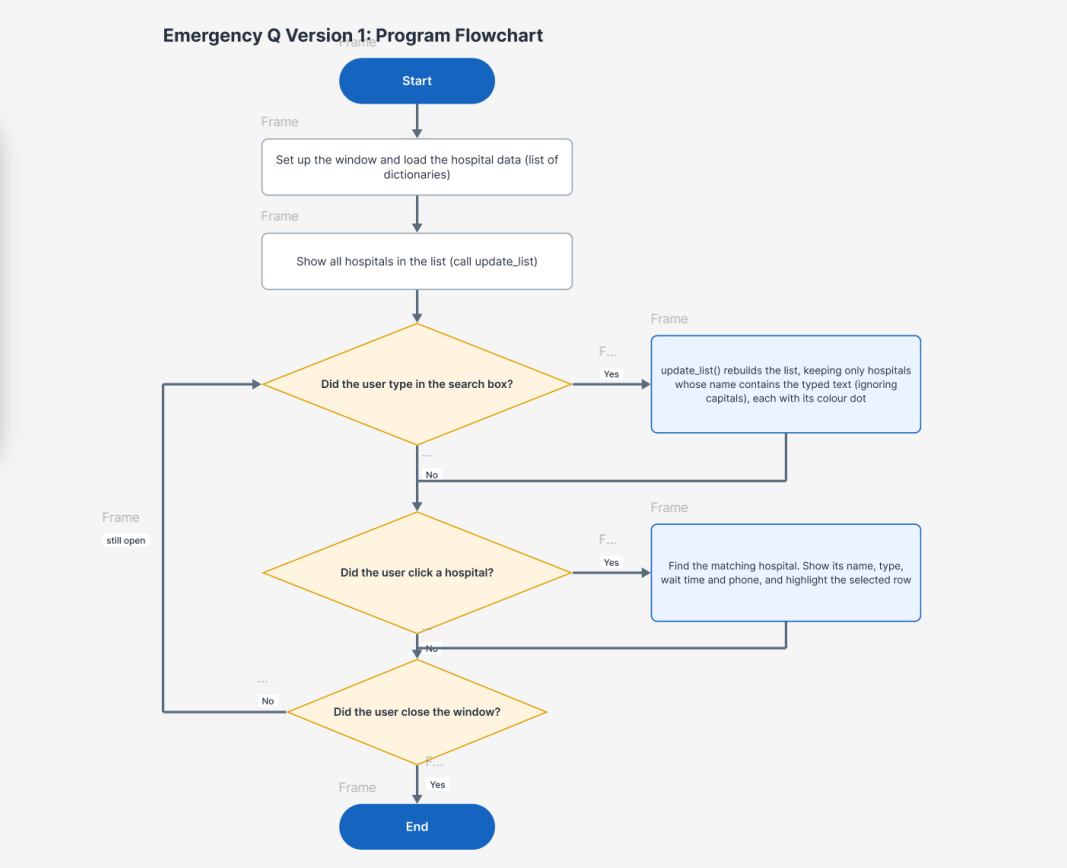
matters because keeping the hospitals as records that the interface just reads is what lets Version 1 grow into the file based, class based, map based final program without me starting over.

11. Table of objects required in the program (Version 1)

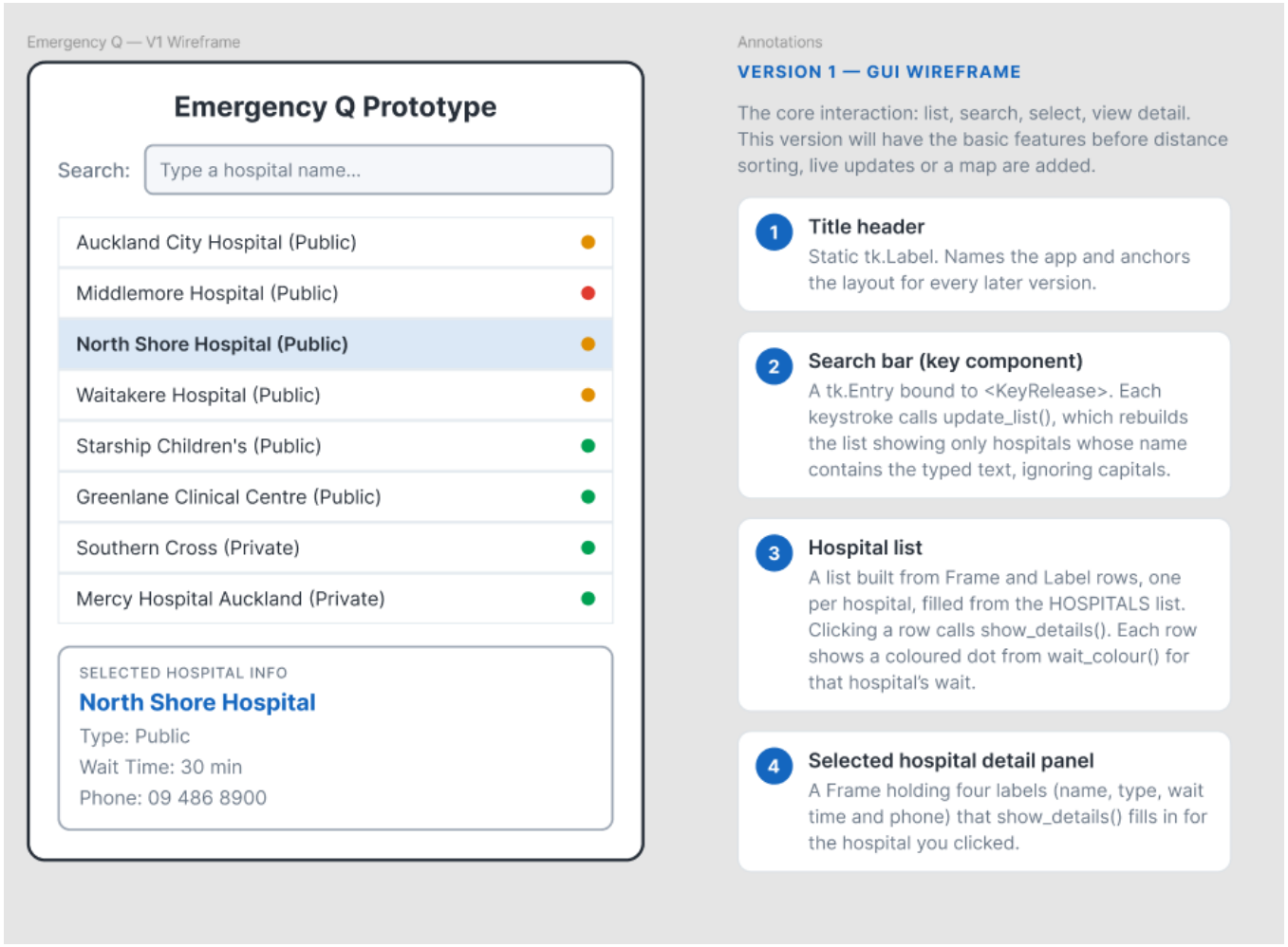
Object / variable / structure	Datatype	Purpose and relevance to the outcome
HOSPITALS	list of dictionaries	The main data. Each dictionary is one hospital with its name, wait, phone and type. A list is used so the program can loop over every hospital to build the display.
hospital record (name, wait, phone, type)	dict of strings	Keeps all the facts about one hospital together in one object. This is the start of the Hospital type I plan to add later.
search_term	string	The text the user types. It is compared, in lower case, against the hospital names to decide what shows.
root / App window	tk.Tk object	The main window that holds and shows every other widget.
search_entry	tk.Entry widget	The box where the user types a hospital name to filter the list.
listbox	tk.Listbox widget	Shows the filtered hospitals and lets the user pick one.
name_lbl, type_lbl, wait_lbl, phone_lbl	tk.Label widgets	The labels in the detail panel that show the selected hospital's information.
selection	tuple of index	The row the user has selected in the listbox, used to look up which hospital to show.
SHORT, MEDIUM (from V2)	integer constants	The wait thresholds (30 and 90 minutes) that decide the green, orange or red colour, kept as constants so they are easy to change.
SUBURBS (from V2)	dictionary of name to (lat, lng)	Maps a suburb name to coordinates for the distance sort. A dictionary is used for quick name lookup.

Version 1:

Screenshot of Version 1 Flowchart:



Screenshot of Version 1 wireframe:



Feedback on Version 1: (testing video is in the zip folder)

Stakeholder	Feedback
SH1	Teodor: There should be a distance calculator for users so that they can see how far away the hospital is, as the wait time of the hospital might not be that long, but it could be a 40 minute drive away from them.
SH2	Sheroy: For users that like really need to go to the E.R have some information saying where to call (111) maybe like a button. Maybe also show how many patients are actually in the waiting room to solve any insecurities in the user's mind.
Teacher's Feedback	Please go back to your Inquiry standard and see what you proposed. Looks like you lost the plot. Good to revisit your old documents to push you back on track.

Summary of Feedback and intended changes to make in Version 2:

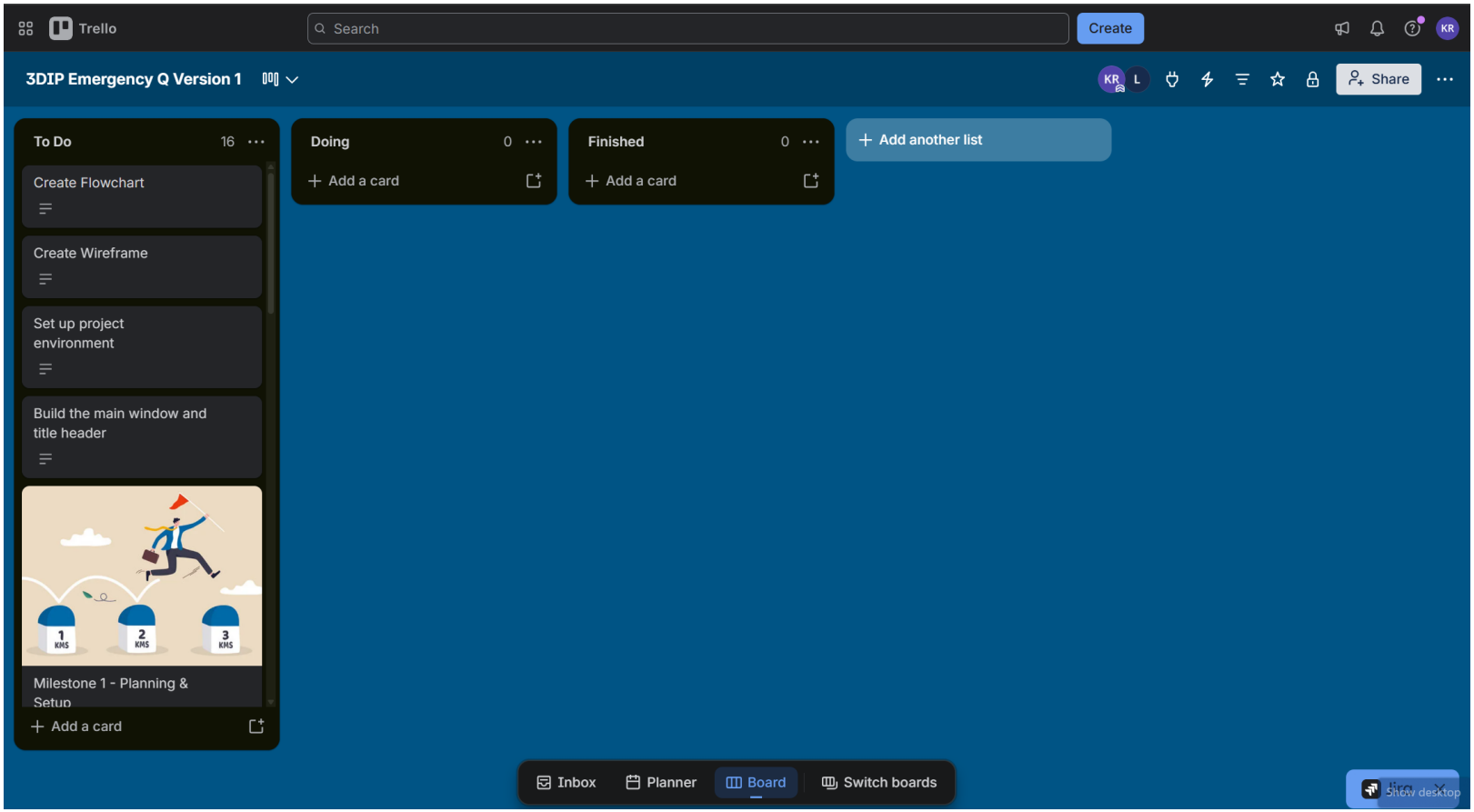
All three pieces of feedback pointed in the same direction: while Version 1 showed that the core interaction worked, it lacked the information someone would actually need to decide which hospital to go to.

Teodor's feedback was the most significant because it revealed a flaw in the underlying idea of Version 1, not just a missing feature. A hospital showing a 10-minute wait is meaningless if it takes 40 minutes to get there — the number on the screen only represented part of the decision. To address this, Version 2 introduces distance. The user selects their suburb from a dropdown, the coordinates are pulled from suburbs.txt, and the Haversine formula calculates the distance from that suburb to each hospital. Each row now displays the distance in kilometres alongside the wait time, and a "sort by nearest first" option allows the list to be organised by distance rather than wait time. I chose a suburb dropdown instead of a free-text address input because free text would require an online geocoding service, and I wanted the program to function without an internet connection.

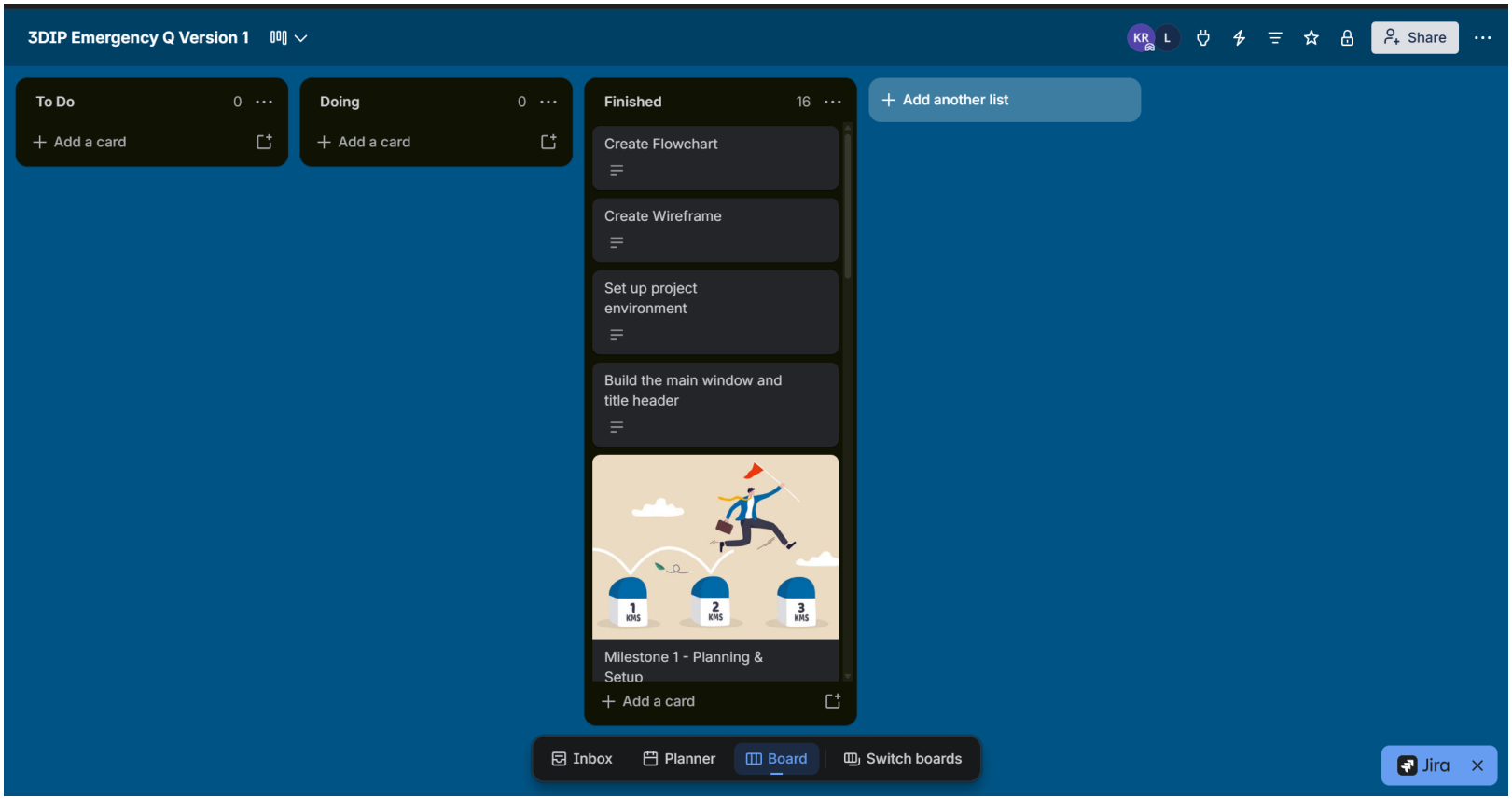
Sheroy's feedback focused on trust rather than adding new features. The first issue was safety: in a real emergency, a person should not be relying on a wait-time app at all. In Version 2, a permanent banner is placed at the top of the screen stating that the app is for informational purposes only and not medical advice. Alongside this is a red "Call 111" button, which opens a warning dialog telling the user to call 111 immediately. This banner remains visible at all times and does not scroll away. The second issue he raised was the lack of confidence in the wait time itself — on its own, it is just an abstract number with no clear basis. To fix this, the details panel now shows the number of patients currently waiting and clearly labels the figure as "time from arrival to triage," making it explicit what is being measured. A "last updated" timestamp was also added to further support trust in the data.

My teacher's feedback highlighted that I had moved away from what I originally proposed in the 91900 inquiry. I revisited that report and compared it directly with what Version 1 actually included. This comparison showed that several elements I had originally committed to were missing: patient counts, a clear explanation of what the wait time represents, emergency contact information, and the search icon. All of these have now been included in Version 2. Since then, I have kept the inquiry report open alongside the code to ensure that the final outcome is evaluated against what I originally planned, rather than what was easiest to implement.

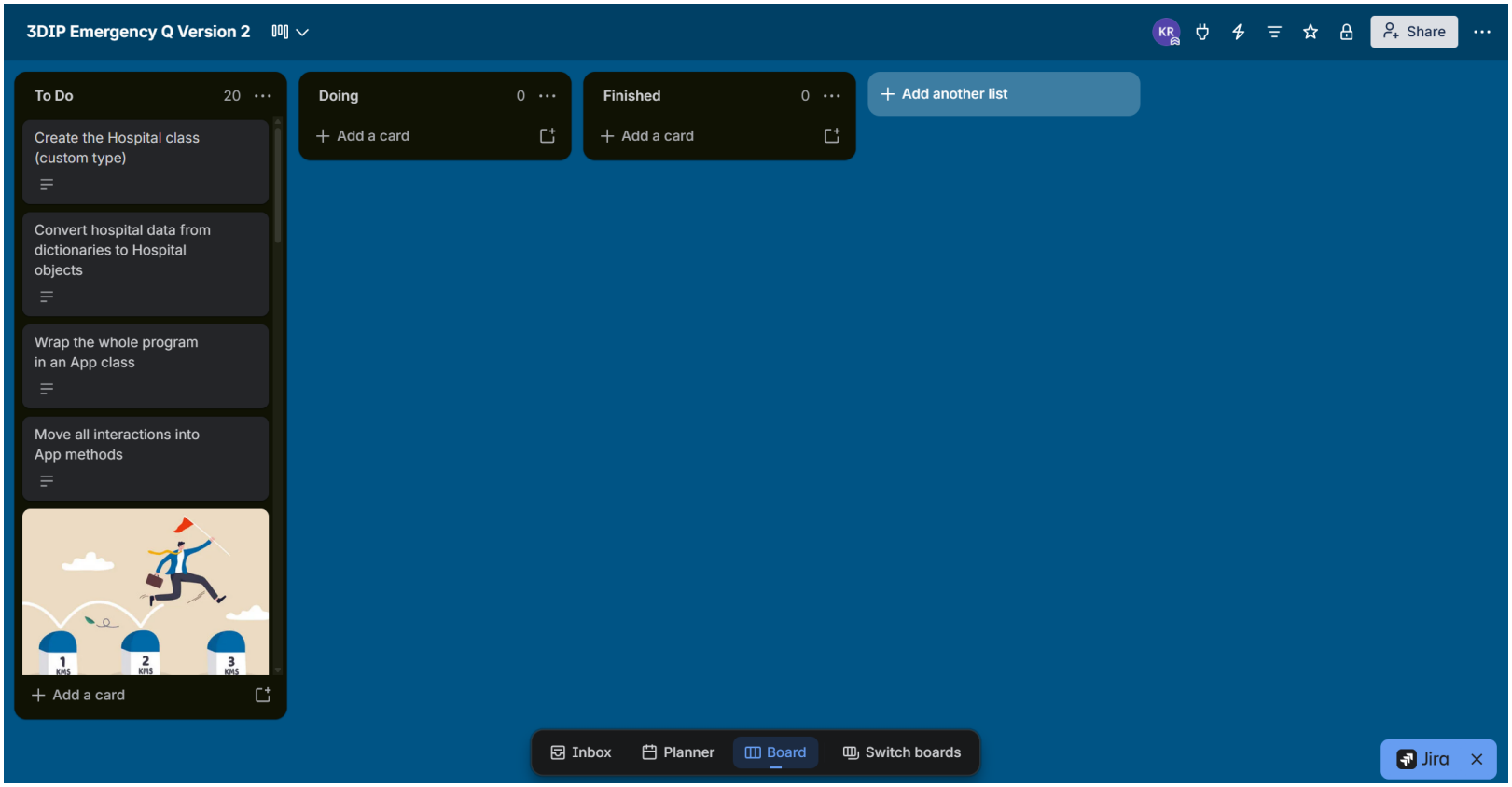
Trello Tasks in version 1 created:



Trello tasks in version 1 completed:



Version 2 Trello created:

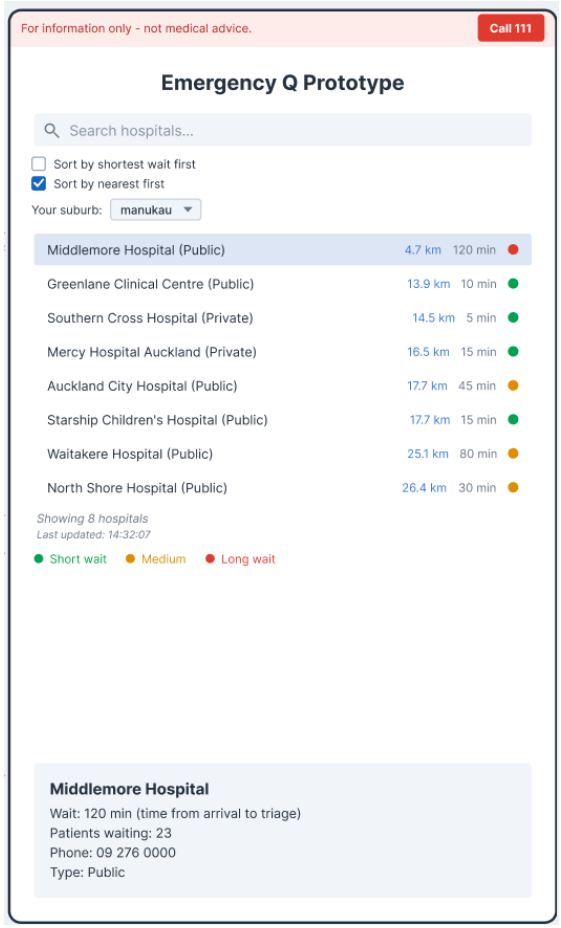


Version 2: Explain what actions will happen in your second iteration.

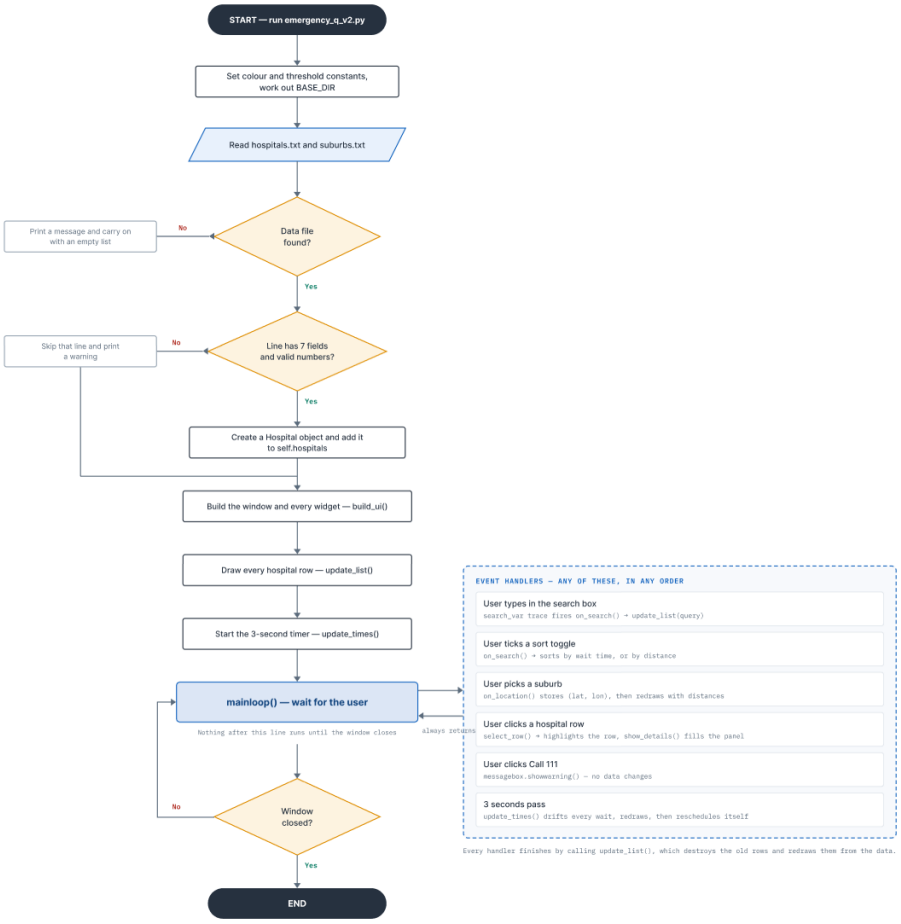
Version 2 builds on the core interaction from Version 1 by adding the real-world information people need to make decisions:

1. Distance calculation: The user selects their suburb from a dropdown, and the Haversine formula calculates the distance from that suburb to each hospital. Each row shows the distance in kilometres.
2. Distance sorting: A "sort by nearest first" checkbox reorders the list by distance rather than wait time.
3. Wait time sorting: A "sort by shortest wait first" checkbox reorders the list by wait time.
4. Patient count: The details panel now shows how many patients are waiting, giving context to the wait time.
5. "Last updated" timestamp: Shows when the data was last refreshed, building trust in the information.
6. Safety banner: A permanent banner stating "For information only - not medical advice" with a red "Call 111" button that opens a warning dialog.
7. File-based data: Hospitals and suburbs are read from text files instead of being hard-coded, making the data easy to update.
8. Live wait time simulation: Every 3 seconds, each hospital's wait time drifts up or down by a random amount, simulating a live feed.
9. Class-based structure: The Hospital class models each hospital with its data and behaviour, and the App class holds the interface.

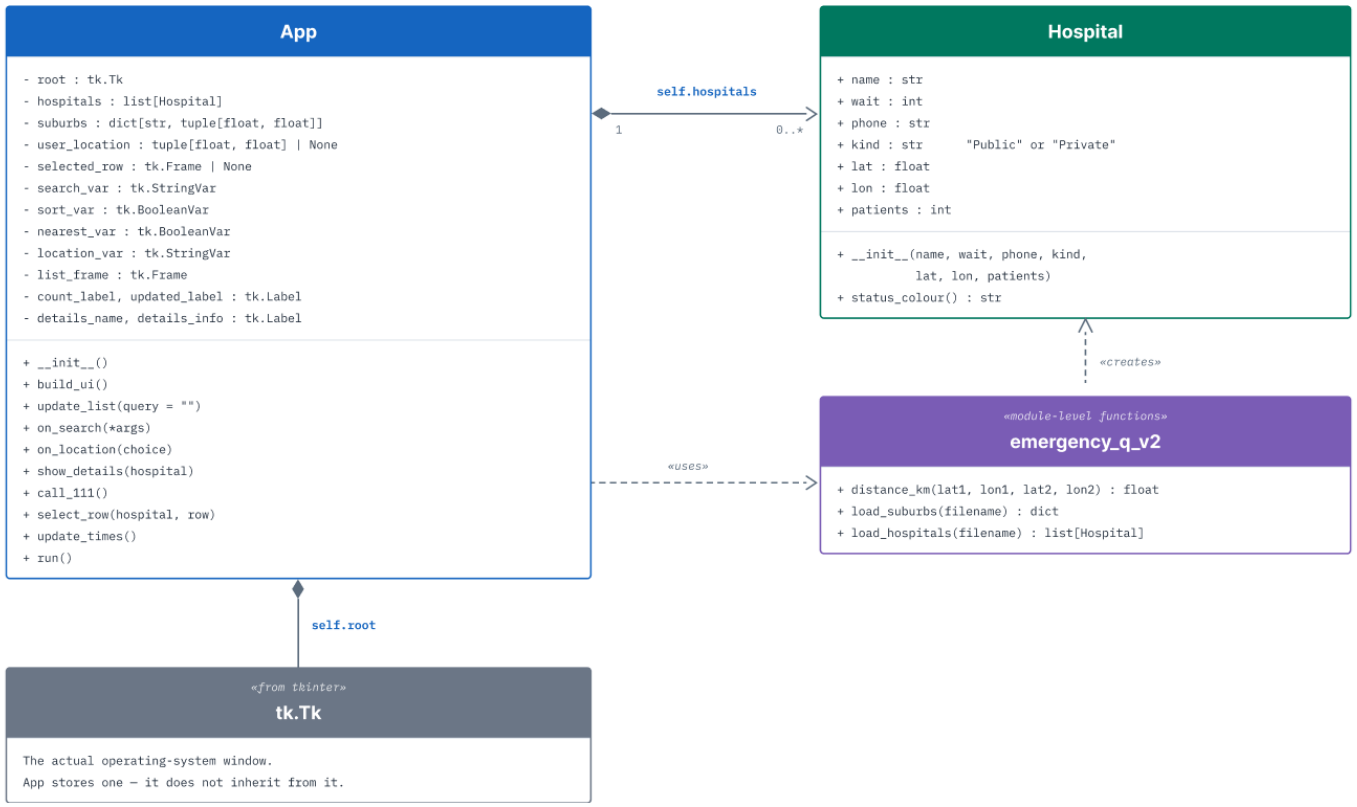
Screenshot of V2 Wireframe:



Screenshot of V2 Flowchart:



Version 2 UML Diagram



Feedback on Version 2: (testing video in the zip folder)

Stakeholder	Feedback
SH1	Teodor: This is good, I like the distance showing, but it would be nice if the user could visualise how far they are from the hospitals. A map would be good.
SH2	Sheroy: I like the “111” button you added, but I would like if the app was a little more visually appeasing and maybe even a map for users to see hospitals live. I think everything is a bit cramped right now.
Teacher’s Feedback	**V2 Feedback 27-8-26:** • Your version 2 is well designed. It makes more sense now and has many usability features. • Good to see you have learnt some new skills such as Haversine mathematical formula to trace co ordinates and you have readily used it. • This app is purely a checking app and is not requiring any inputs. It is just an informative app. • One great feature in your app is reducing the time of user in case they dont want to enter the name of hospital manually. Your program still accepts both clicking the hospital and also the empty input. • Sort feature is another highlight. • May be increase the font size slightly.

Summary of Feedback and intended changes to make in Version 3:

Summary of Feedback and Intended Changes to Make in Version 3

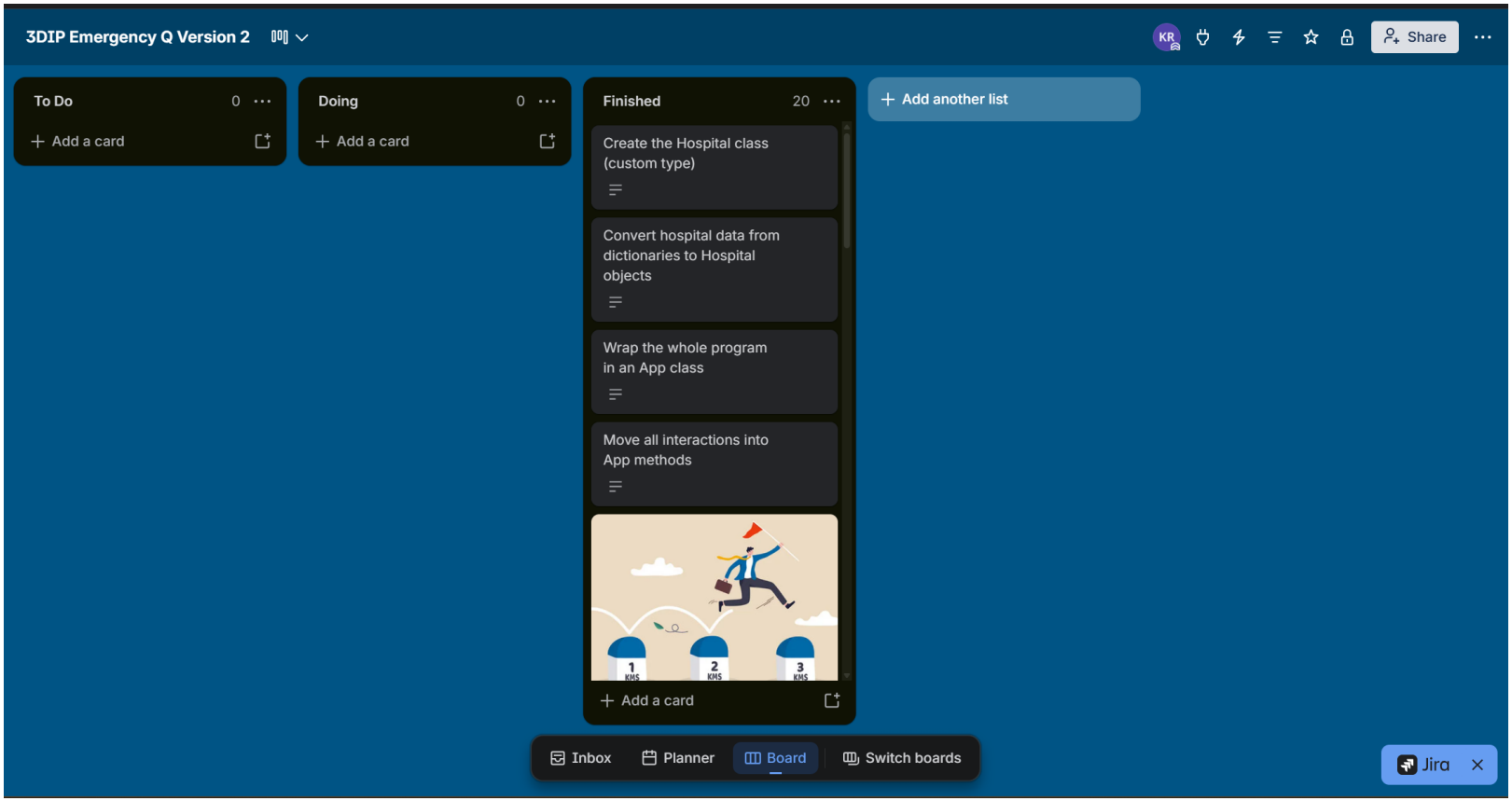
The feedback from Version 2 confirmed I was on the right track, but also highlighted areas where the current implementation could be improved. I agree with all three pieces of feedback and will carry these insights forward into any future development.

Teodor's feedback about needing a map was completely right. In Version 2, I showed the distance in kilometres next to each hospital, but that only tells half the story. A number like "12.3 km" doesn't tell the user which direction to travel, which hospitals are clustered together, or what the spatial relationship actually looks like. I added an interactive tkintermapview map in Version 3 to address this, but Teodor is right that even this implementation has limitations. The map relies on OpenStreetMap tiles which require an internet connection, and the pins are manually placed rather than using a live GPS feed. It gives the user a visual sense of location, but it's not as accurate or responsive as something like Google Maps would be. For the prototype, the tkintermapview solution was the best option because it's free, doesn't require an API key, and avoids the privacy implications of collecting real GPS data. But Teodor's point is valid - in a real-world deployment, a more sophisticated mapping solution would be needed.

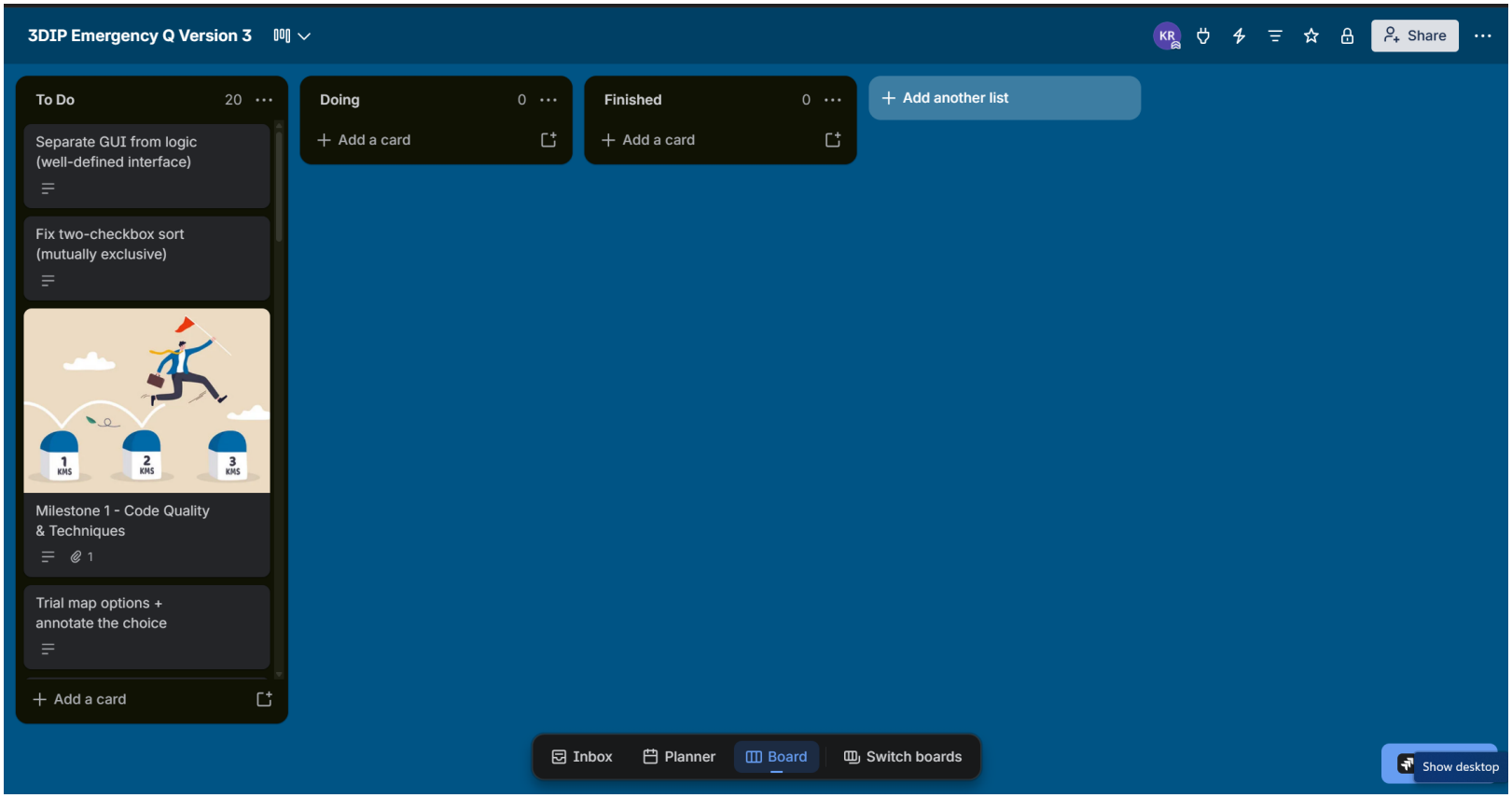
Sheroy's feedback about visual appeal and professionalism was also something I agree with. He was right that the single-page layout of Version 2 felt cramped and cluttered, especially on smaller screens where the list and details panel competed for space. In Version 3, I restructured the app into three separate pages to address this. However, Sheroy is right that there's still room for improvement - the interface is clean but could benefit from more visual hierarchy, better use of icons, and perhaps a more polished colour scheme. The current design is functional and professional enough for a prototype, but a production version would need a dedicated UI designer to make it truly polished. His feedback about the 111 button actually calling 111 is also valid - the current implementation opens a warning dialog because Tkinter can't place phone calls, but a real app should integrate with the device's phone system to actually dial the number.

My teacher's feedback about increasing the font size was something I had already addressed in Version 3, and the feedback confirmed I made the right call. The teacher was right that the font was slightly too small in Version 2, and the larger fonts and dedicated detail page with the "hero" wait time make the app much more readable at a glance. However, there's still room for improvement - in a future version, I could add a high-contrast mode or a font size slider to accommodate users with different visual needs.

Version 2 Trello completed:



Starting on Version 3 Trello:



Iteration3: Explain what actions will happen in your 3rd iteration.

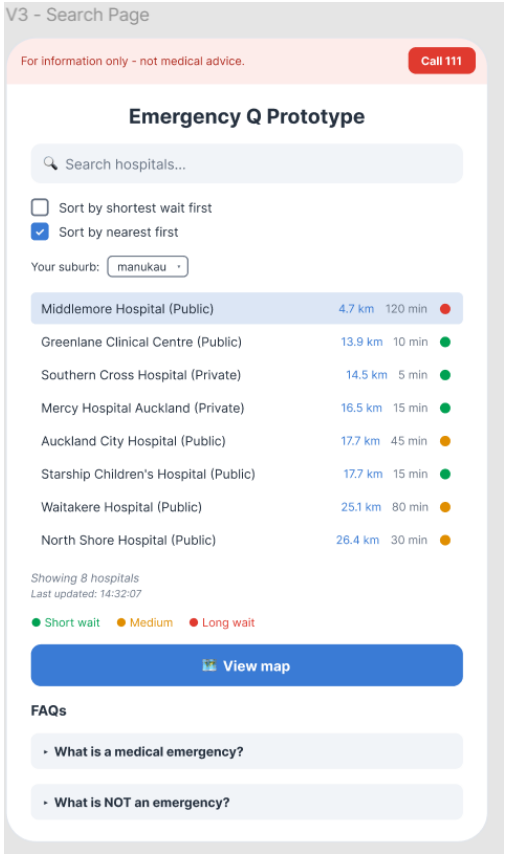
Version 3 transforms the single-page app into a professional multi-page interface with an interactive map. The key additions are:

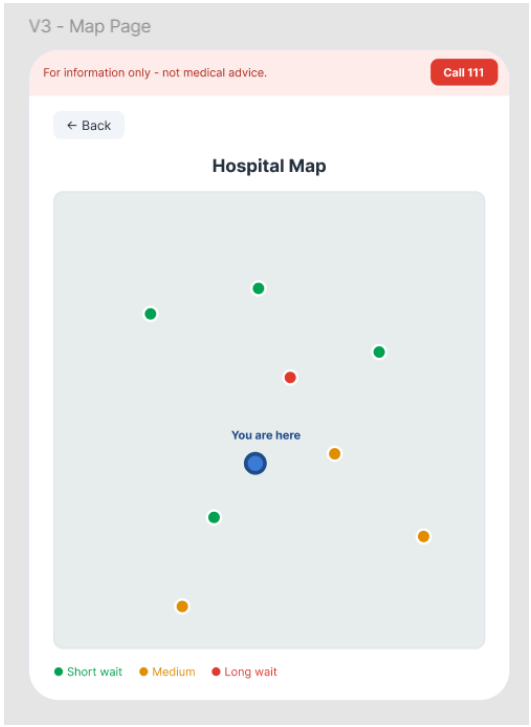
1. Multi-page navigation: Three stacked pages (search, map, detail) switched using .tkraise() with a back button that remembers where to return.
2. Interactive map: An OpenStreetMap-based map with colour-coded pins for each hospital, showing green/amber/red status based on current wait times. When a pin is clicked, the detail page opens with that hospital's information.
3. User location pin: When a suburb is selected, a blue "you are here" pin drops on the map and the map centres on that location, visualising the distance to each hospital.
4. Large detail page: A dedicated detail page with a big colour-coded wait time (the "hero" element), making the most important information immediately readable.
5. Collapsible FAQs: Frequently asked questions that expand when clicked, providing helpful context without taking up permanent screen space.
6. Mutually exclusive sorts: The "sort by shortest wait" and "sort by nearest" checkboxes now disable each other, removing ambiguity.
7. Reusable components: A build_banner() function reused on all three pages and a build_faq() function for each FAQ item, eliminating duplicate code.

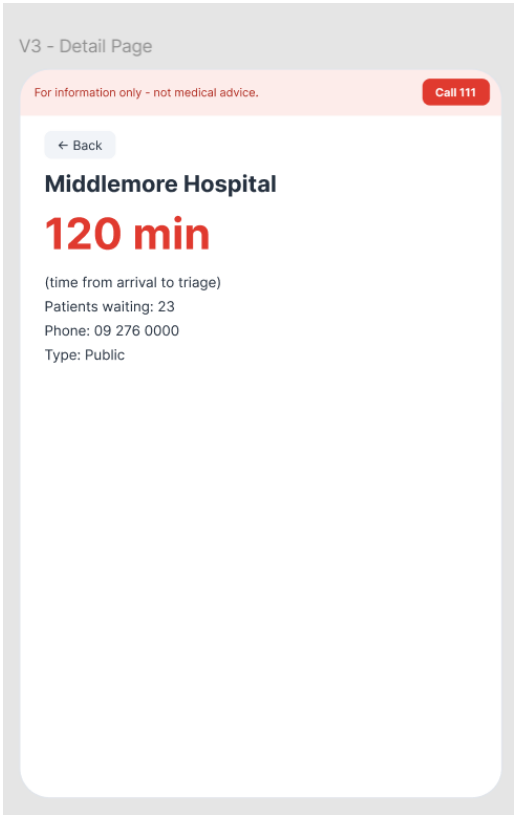
Screenshot of V3 Flowchart:

https://www.figma.com/board/NnRUX1By8wNwbUvvqlzyQP/Emergency-Q-V3---Program-Flowchart?node-id=0-1&t=SfcKT8iJAxKHb6bF-1

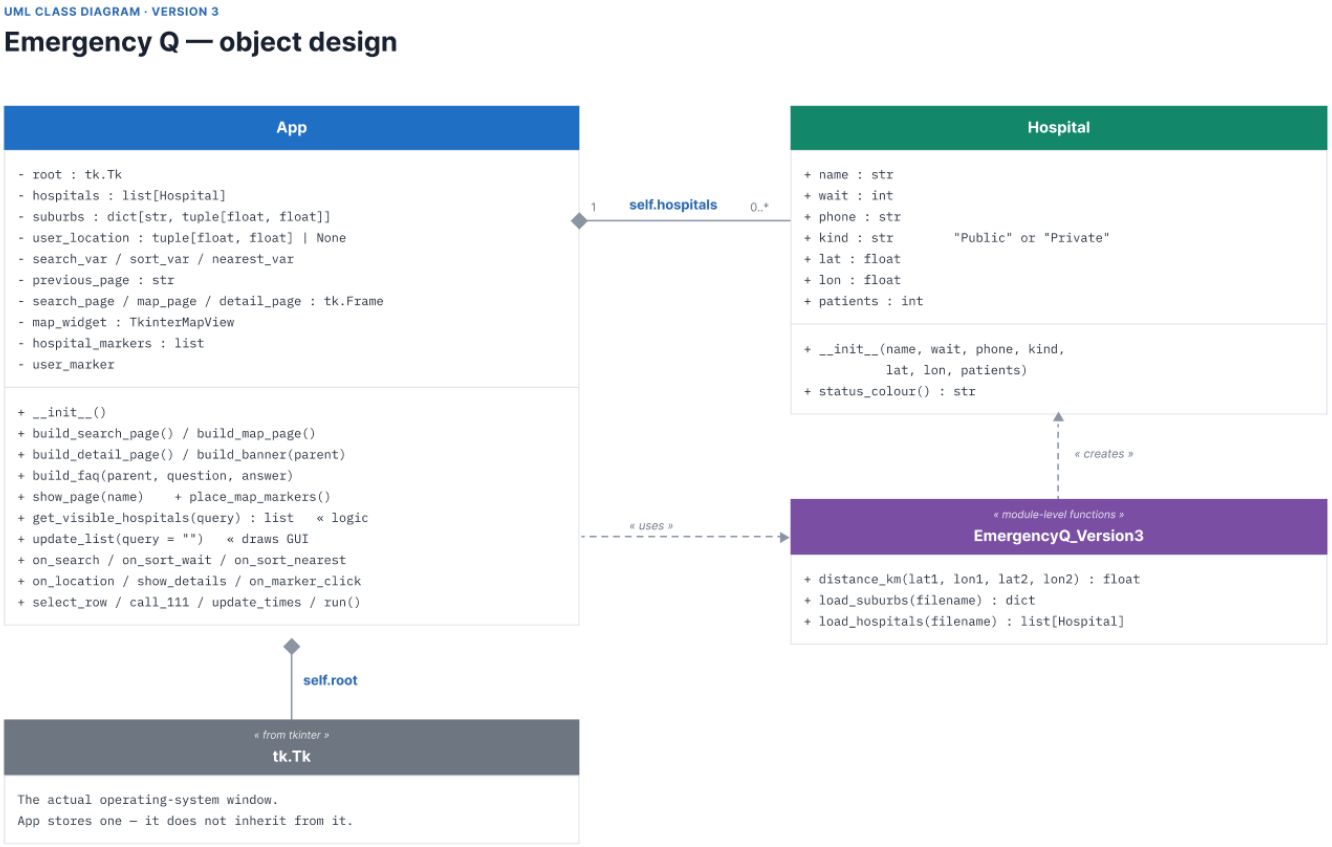
Screenshots of V3 Wireframe:


V3 - Search Page
For information only - not medical advice.
Call 111
Emergency Q Prototype
Search hospitals...
Sort by shortest wait first
Sort by nearest first
Your suburb: manukau
Middlemore Hospital (Public) 4.7 km 120 min
Greenlane Clinical Centre (Public) 13.9 km 10 min
Southern Cross Hospital (Private) 14.5 km 5 min
Mercy Hospital Auckland (Private) 16.5 km 15 min
Auckland City Hospital (Public) 17.7 km 45 min
Starship Children's Hospital (Public) 17.7 km 15 min
Waitakere Hospital (Public) 25.1 km 80 min
North Shore Hospital (Public) 26.4 km 30 min
Showing 8 hospitals
Last updated: 14:32:07
Short wait
Medium
Long wait
View map
FAQs
What is a medical emergency?
What is NOT an emergency?


V3 - Map Page
For information only - not medical advice.
Call 111
← Back
Hospital Map
You are here
Short wait
Medium
Long wait


V3 - Detail Page
For information only - not medical advice.
Call 111
← Back
Middlemore Hospital
120 min
(time from arrival to triage)
Patients waiting: 23
Phone: 09 276 0000
Type: Public

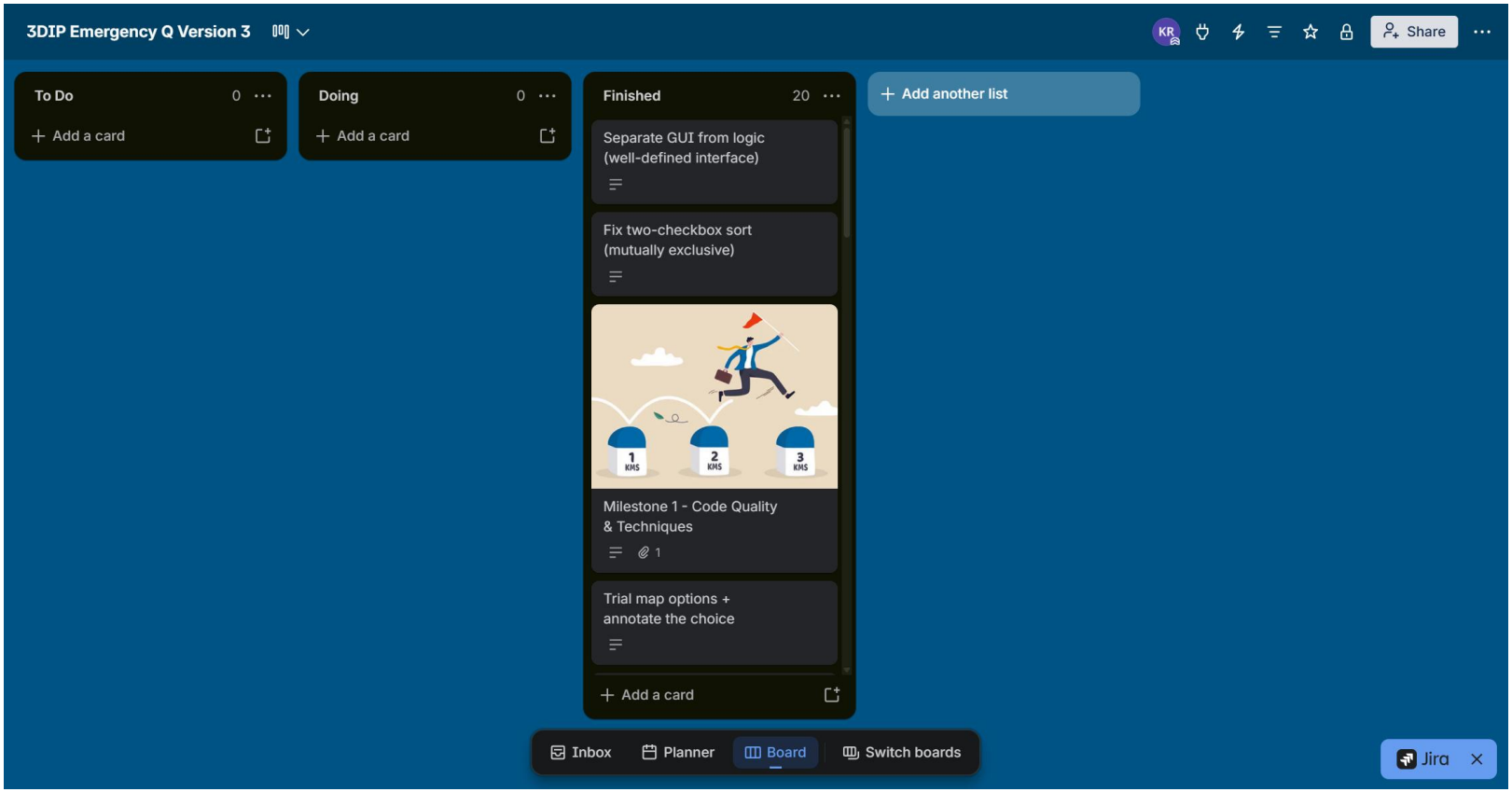
Screenshot of V3 UML Diagram



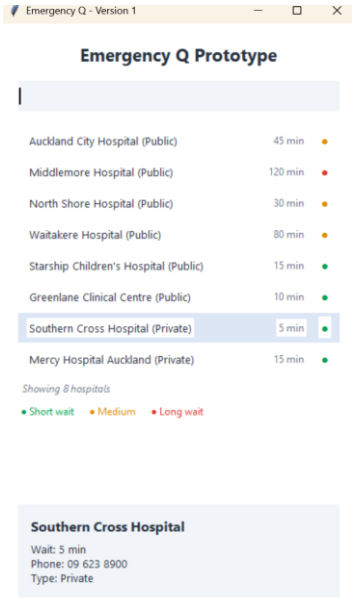
Feedback on Version 3: (testing video in the zip folder)

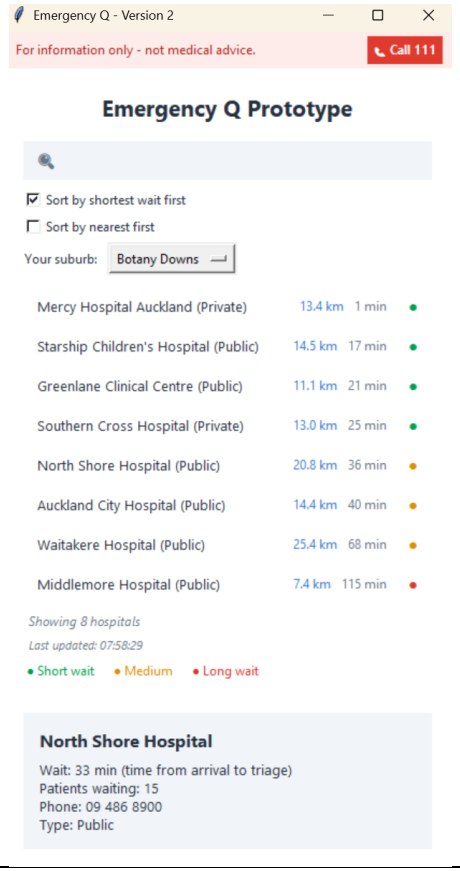
Stakeholder	Feedback
SH1	Sheroy: I like the multi-page setup and how everything is laid out, however, I think that you could've added some more images or information to the details page. I like the simplicity of it but I think a bit more information might be helpful. I also think that the 111 button should actually call 111.
SH2	Teodor: I like the map view, I think it is very professional, however for people that are really in an emergency, they need accurate maps, that can track their location and give them directions to the hospital. Maybe integrating google maps or something?

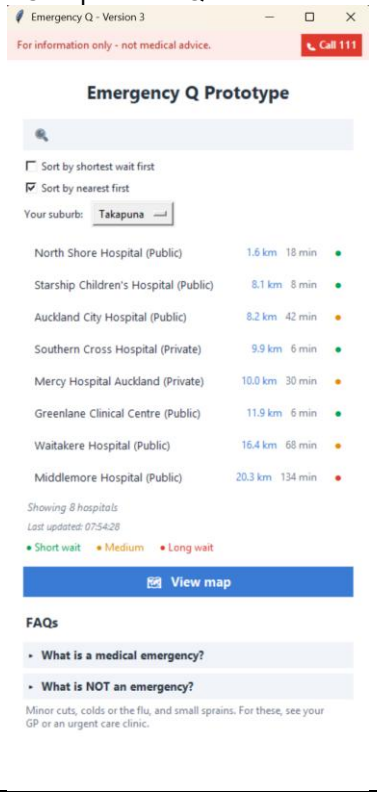
Version 3 Trello Completed:



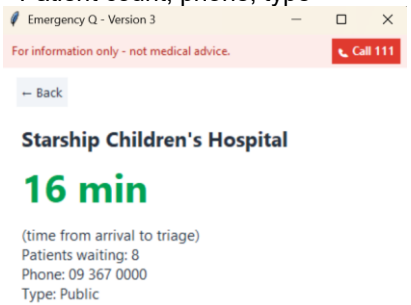
A comparative chart of Version GUI showing iterative improvement

Version1 Description	GUI Design
Description: Basic single-page interface showing a hard-coded list of Auckland hospitals with a search filter and detail panel. No distance information, no safety features, no file reading. Strengths: Simple, works, core interaction functional. Proves the concept. Weaknesses: No distance info, no patient counts, no 111 button, data hard-coded, single page feels cramped and unprofessional. User Feedback: "Need to see distance" (Teodor), "Need 111 button and patient counts" (Sheroy), "Lost the plot from your inquiry" (Teacher).	Features: • Title heading • Simple search box • Hospital list with colour-coded dots • Colour legend • Detail panel at bottom showing selected hospital Layout: Single scrolling page, all content visible at once 
Version2 Description and Feedback	**Gui Design**
Description: Enhanced single-page interface with distance calculation, sorting, patient counts, safety banner, and file-based data. Hospitals and suburbs loaded from text files. Live wait time simulation every 3 seconds.	Features: • Safety banner with 111 button • Search box with magnifying glass icon • Sort checkboxes (shortest wait / nearest)

Strengths: Much more informative, safe, and robust. Distance sorting is a major improvement. Patient counts and last updated timestamp build trust. Weaknesses: Still feels cluttered on one page, no map, visual design needs polish, font slightly small. User Feedback: "A map would be good" (Teodor), "Make it more visually appeasing" (Sheroy), "Increase font size slightly" (Teacher).	• Suburb dropdown selector • Hospital list showing distance + wait time • "Last updated" timestamp • Improved detail panel with patient counts Layout: Single scrolling page with permanent banner, list area, and details panel at bottom 

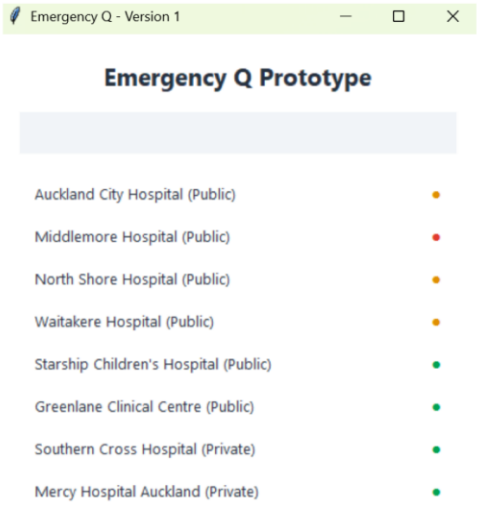
Version 3 Description and Feedback	GUI Design
Description: Multi-page professional interface with interactive map, dedicated detail page, collapsible FAQs, and refined visual design. Clean separation of logic and GUI. Map pins recolour live with wait times. Strengths: Professional, uncluttered, informative, interactive, visually polished. Each page has one clear purpose. Map visualises distance effectively. Weaknesses: Requires tkintermapview library installation, map requires internet access for OpenStreetMap tiles. User Feedback: To be gathered after testing.	Search Page: • Safety banner (reused) • Search box with icon • Mutually exclusive sort checkboxes • Suburb dropdown • Hospital list with colour-coded dots • "Last updated" timestamp • View map button • Collapsible FAQs 

Map Page:
• Safety banner
• Back button
• Interactive OpenStreetMap
• Colour-coded hospital pins
• Blue "you are here" pin
• Colour legend

	Detail Page: • Safety banner • Back button • Hospital name (large) • Big colour-coded "hero" wait time • Patient count, phone, type  Layout: Three separate pages with smooth navigation using tkraise()

Emergency Q: Comparative GUI — Iterative Improvement V1 → V2 → V3

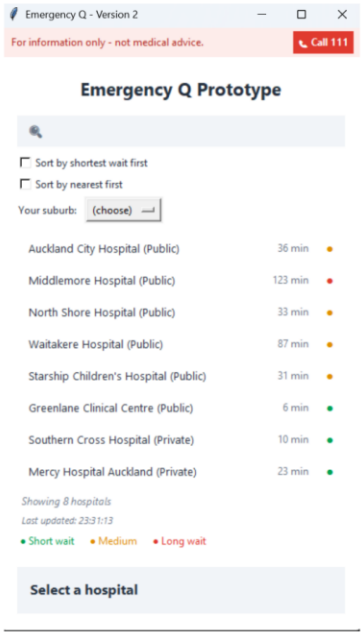
Version 1
Base interaction



Starts the project with:

- Window, title & live search box
- Hospital list with green / amber /
- red status dots
- Click a hospital -> detail panel
- Covers sequence, selection & iteration

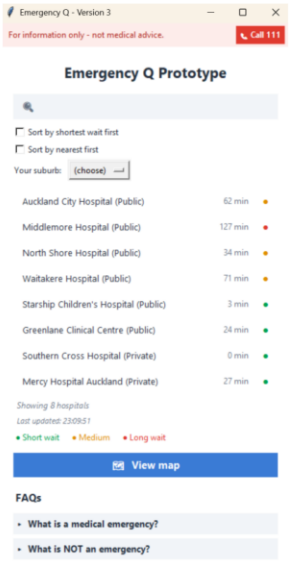
Version 2
Data, distance & safety



New in this version:

- Reads hospitals.txt (file handling)
- Wait time in minutes per hospital
- Sort by shortest wait
- Sort by nearest (Haversine distance)
- Suburb picker -> km distances
- Safety banner + red Call 111
- 'Last updated' live-refreshing times
- App + Hospital classes (OOP)

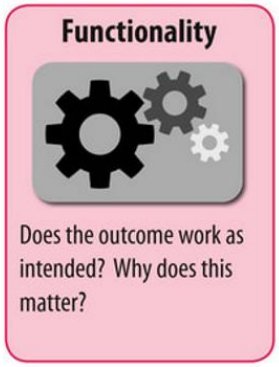
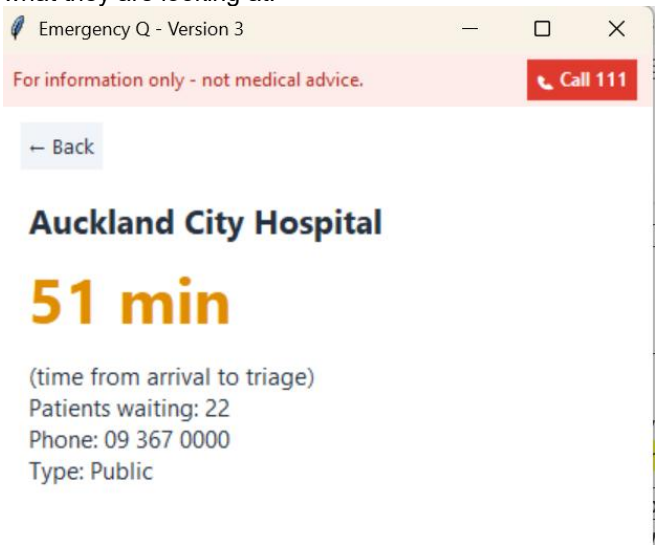
Version 3
Map, multi-page & polish

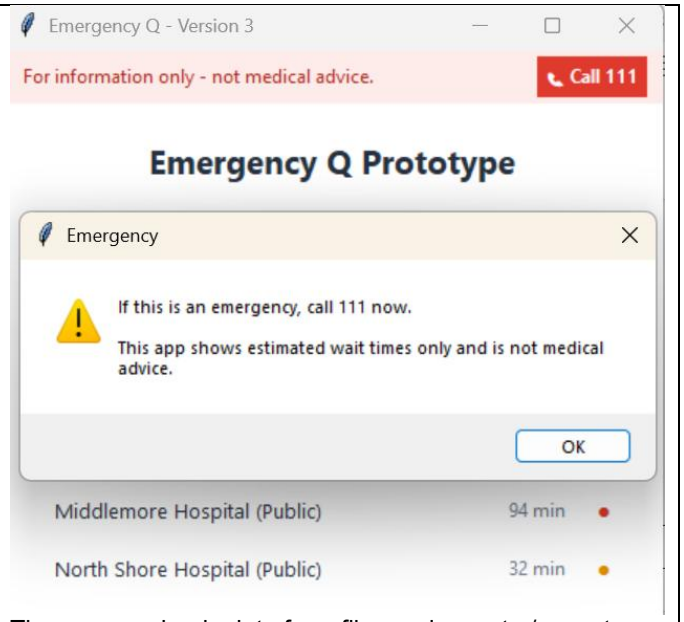


New in this version:

- GUI/logic split (get_visible_hospitals vs update_list)
- Two sorts made mutually exclusive
- Interactive map (tkintermapview)
- Colour-coded, live-recolouring pins
- Multi-page: search -> map -> detail
- Blue 'you are here' suburb pin
- Collapsible FAQs
- Big colour-coded wait on detail page

Discuss how you addressed the Relevant Implications you described and explained earlier.

Relevant Implication	How I applied this in the development of my outcome.
Functionality  	The program reliably shows and sorts hospital wait information, and clearly labels what a "wait time" actually means. In the details panel, the wait is explicitly described as "time from arrival to triage" so users understand exactly what they are looking at.  The safety banner clearly states the app is "For information only - not medical advice," and the "Call 111" button directs users to the appropriate emergency service.

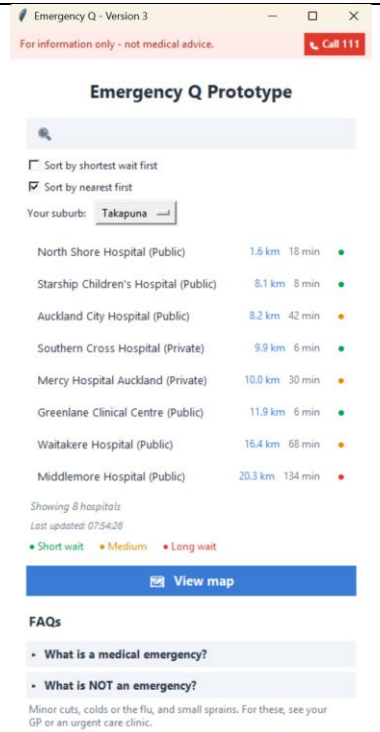
	 The program loads data from files and uses try/except blocks to handle missing or corrupted files gracefully, ensuring the app always opens and functions even if the data has issues. I also used a max(0, ...) clamp when updating wait times so they never go negative, which would be confusing and unrealistic.
Usability and Accessibility	The interface uses clear colour contrast (white background, dark text, distinct status colours).

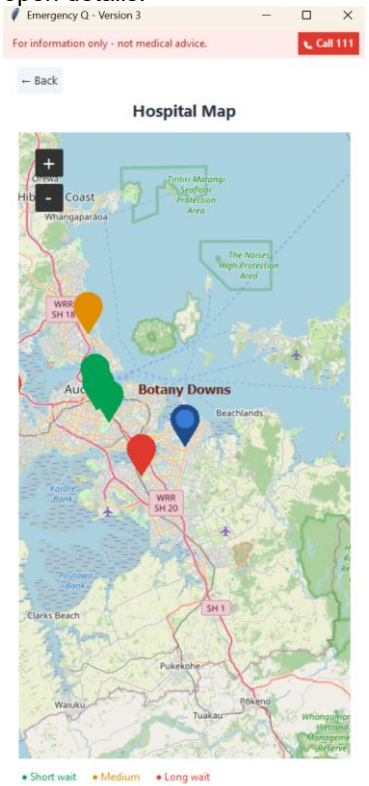
Usability

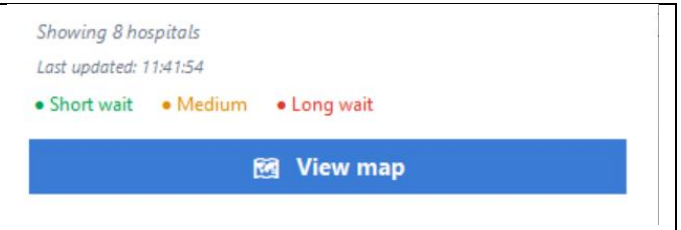
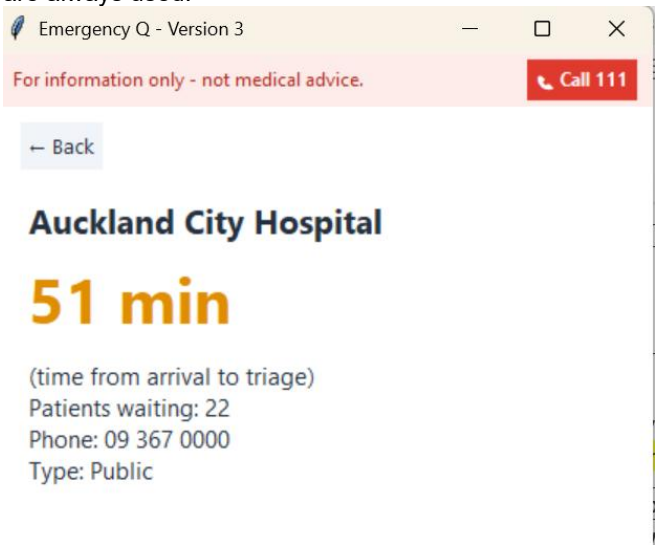
System status, consistency, user freedom, real world match, error prevention / recovery, flexibility etc

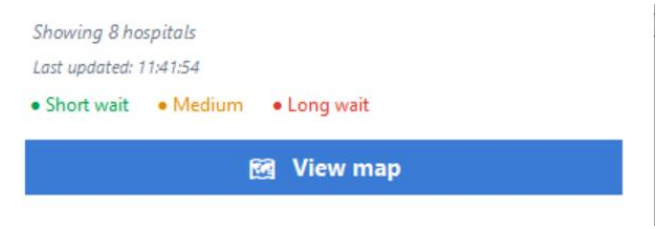
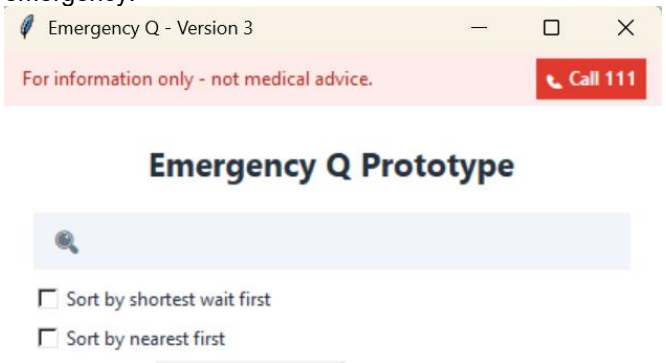
Accessibility

Can all users access the content? Think about older browsers, visually impaired users, small screens.



	Click targets are large enough for easy interaction, especially on the map where markers can be clicked to open details.  The colour status system (green/amber/red) means users don't have to read numbers to understand the situation - they can read the status at a glance.

	 The search box is case-insensitive and uses substring matching so users don't need to type the full hospital name. The detail page has a large "hero" wait time that is immediately visible. Suburb selection is done through a dropdown to prevent typos and ensure valid coordinates are always used. 
Aesthetics	I kept one restrained palette defined as named constants at the top of the program so every part of the screen uses the same consistent colours. Colour is only ever used to

	carry meaning (green/amber/red for wait status), never as decoration.  The one loud element on the screen is the red "Call 111" button, which is the thing that should stand out in an emergency.  Generous spacing and a single font at three sizes keep the list scannable instead of crowded. The multi-page layout reduces visual noise by showing only one screen at a time. In an emergency context, the appearance is doing a functional job, not just a decorative one.
Legal and Copyright	I am only using open-source libraries (Tkinter, tkintermapview) within their licences. Public hospital contact details are from official public sources. The data files are plain text that I created using publicly available information. All code is original and written by me. The program clearly states it is "for information only" and not medical advice, protecting both the user and myself from

Legal - Copyright Act | Privacy Act  **Intellectual Property** copyright, creative commons, public domain, attribution, own work	any liability regarding health decisions made based on the app's data.
Privacy and Data Protection  **Privacy** Does your outcome use / collect user information? How do you keep user data secure? Does your website use cookies?	The program only ever handles totals, meaning an estimated wait and a count of patients waiting, and never anything about an individual patient, so no health information about a person is stored or shown. There is no account, no login, and no personal data collected from the user at all. The suburb the user picks is held in memory for that session only and is never written to a file or sent anywhere, so nothing about their location survives closing the program. No GPS location is collected. This matters under the Privacy Act 2020 and the Health Information Privacy Code 2020.
Future Changes **Sustainability & Future Proofing** - How will your outcome remain up-to-date and relevant?	The data, meaning the hospitals, suburbs and thresholds, will need updating over time. Reading data from files and using named constants means I can update it without rewriting code, which keeps the program flexible. The hospital data is stored in hospitals.txt and suburbs in suburbs.txt, so adding a new hospital or updating a wait time doesn't require touching the Python code. The threshold values (SHORT=30, MEDIUM=90) are constants at the top of the file, so if the clinical definition of "short" wait changes, it's a one-line edit. A big future change would be rebuilding the outcome as a mobile app. The current version runs on a computer using Tkinter, and a laptop can't make phone calls, so the "Call 111" button can only show a warning dialog. On a phone it could actually dial — a mobile app opens the phone's dialer through a tel: link, so tapping "Call 111" would start the call in one tap. Rebuilding it for phones would mean

	swapping Tkinter (which doesn't run on iOS or Android) for a mobile framework such as Flutter, React Native, or a progressive web app. A phone would also provide real GPS, so "you are here" and "sort by nearest" would use the user's actual location instead of a suburb dropdown. The reason this matters: people decide whether something is an emergency on their phone, so a mobile version could let them act in the moment instead of only being told to call 111. Another change needed to make the app real is getting the actual wait-time data. At the moment the wait times are simulated and drift randomly for the prototype. Real data would have to come from Health New Zealand (Te Whatu Ora), who run the public hospitals and hold the ED information, through a formal data-sharing agreement and a live feed from each hospital's system. This would require government intervention because the data isn't publicly available, releasing live ED numbers is a policy decision, it falls under the Privacy Act 2020 so no individual patient can be identifiable, and there would need to be an agreed standard for accuracy and responsibility if a wait time was ever wrong. Queensland Health already publishes an official ED wait-times dashboard, which proves a government health body can share this data safely and is the model New Zealand could follow. This is the biggest limitation, because the programming can be done by a student but the data can only come from the institutions that hold it.
Equity	An app that only works on a phone or computer can leave out people without devices or internet. Keeping it light and dependency-free early on (Version 1 and 2 require no internet), and keeping the interface simple enough for a family member to check for someone else, lowers that barrier. The desktop app can run on older computers that might not handle modern web apps. The wider idea is also meant to run as a website for even greater accessibility.

	The text is readable and the interface is simple enough that it can be used by people who aren't tech-savvy.

Fitness for Purpose:

How Planning, Testing and Trialling Led to a High-Quality Outcome:

Planning: The Trello board broke the work into small, testable cards. The wireframes and flowcharts set the design before any code, so versions grew without rewrites. Each version had a clear purpose: V1 proved the core interaction, V2 added decision-making information, V3 polished the interface.

Testing: The testing tables drove real fixes. The boundary tests at 29/30 and 89/90 minutes confirmed the colour thresholds worked correctly. The invalid data tests (missing file, bad lines, non-number waits) proved the loader was robust. The integration tests caught the stale pin bug in V3.

Trialling Components: I trialled different approaches before committing:

- Search: Tried a dropdown vs. a free-text search box → chose free-text for flexibility
- List display: Tried a Listbox vs. custom card frames → chose custom frames for colour control and layout
- Map: Considered static image vs. Google Maps vs. tkintermapview → chose tkintermapview for free, interactive, no-API-key mapping
- Data storage: Hard-coded list vs. file-based → chose file-based for updatability

How it led to quality: Every version added a tested improvement that answered real feedback. V1 showed it worked. V2 made it useful. V3 made it professional. The outcome is reliable, usable, and fit for the brief.

Future Development

The program could be further developed in several ways:

1. Live API feed: Instead of simulating wait times with random.randint(), connect to a real hospital data API (if one becomes available).
2. Web version: A web-based version would be accessible from any device with a browser, improving equity.
3. Accessibility: Add screen-reader support and a high-contrast mode.
4. Real GPS with consent: Add optional GPS location with user consent and a clear privacy policy.

5. Directions: Integrate with a mapping service to provide turn-by-turn directions to the selected hospital.
6. Notifications: Allow users to set alerts for when a hospital's wait drops below a threshold.

Describe two alternative approaches you considered.

Explain why you rejected them.

Alternative 1: Static Map Image vs Interactive Map

What I considered: Instead of using tkintermapview for an interactive map, I considered using a static image of Auckland with hospital locations marked as fixed dots.

Why I rejected it: A static image would not support panning, zooming, or clicking on hospitals to see details. The pins would be fixed and could not be recoloured when wait times changed. Since the whole point of the map was to *visualise* live data, a static image would defeat the purpose. The interactive map with live pins is a much better fit for the program's goals.

Alternative 2: Free-Text Address Input vs Suburb Dropdown

What I considered: Instead of a dropdown list of suburbs, I considered letting users type their full address and using an online geocoding service (like Google Maps API) to convert the address to coordinates.

Why I rejected it: A free-text address input would require an internet connection and an API key (which would involve billing and potential privacy issues). The suburb dropdown works entirely offline and avoids collecting detailed location data. For a prototype, the suburb dropdown is sufficient and more privacy-friendly. If the app were to be developed further, I would add optional GPS location with explicit user consent.

Describe One Major Programming Challenge You Encountered

Challenge: Getting the Distance to Show After Suburb Selection

What caused it: After adding the distance feature in Version 2, the kilometre distances only appeared after I typed something in the search box. The on_location() method stored the suburb coordinates but didn't refresh the list. update_list() only ran when the search text changed or when the live timer triggered a redraw, so the bug was hidden whenever a search was active. If the user picked a suburb and didn't type anything, the distances would never appear.

How I investigated it: I noticed that distances showed up only after some other action triggered a redraw - like typing a letter into the search box or waiting for the live timer to update. This suggested that on_location() was storing the data correctly, but nothing was telling the list to redraw. I traced through the code and confirmed that on_location() set self.user_location but never called update_list() to refresh the display.

How I solved it: I added a call to update_list() inside on_location() so that picking a suburb redraws the list immediately. This was fixed in Commit 8. The evidence is in my testing table: row 1.15 was recorded as a fail because distances didn't appear after suburb selection. Row 1.16 is a pass - Takapuna showed North Shore as the nearest at 1.6 km immediately after selection.

What I learnt: This challenge taught me that an event handler has to trigger its own redraw - storing data isn't enough, the interface has to be told to update. I also learnt that a bug can stay hidden because another code path masks it. In this case, the live timer redrew the list every three seconds, so the distance would eventually appear even without the fix, but a user shouldn't have to wait for an arbitrary timer to see the information they just asked for. A good user experience means the interface responds immediately to user actions, and that requires explicit redraw triggers.

Choose one important function or algorithm from your program.

Function: get_visible_hospitals(query)

Why it was designed this way: It separates the LOGIC (filtering and sorting) from the GUI (drawing). get_visible_hospitals() does the thinking and returns a list; update_list() does the drawing. This is a clean separation of logic from display. The logic is isolated, so it can be tested independently without needing to create any GUI widgets.

Why this solution is efficient: One pass to filter the hospitals ([h for h in self.hospitals if query in h.name.lower()]) and one sort. The function doesn't touch any widgets, so it's fast even with many hospitals. The filtered, sorted list is returned once, and the GUI can draw from that list without redoing the logic.

What would happen if it were written differently: If I had mixed the filtering and sorting INTO the drawing code (as V1 and V2 did), the logic would be coupled to the GUI. That would make it impossible to unit-test the logic without creating a full Tkinter window. It would also make the drawing

code more complex and harder to read. The current design keeps each part to one job: get_visible_hospitals handles the logic, and update_list handles the display.

Explain how decomposition helped you during development.

Functions

Standalone functions like distance_km(), load_hospitals(), and load_suburbs() are independently testable and reusable. They don't depend on the GUI or any global state, so they can be tested in isolation. distance_km() is pure: given the same inputs, it always returns the same output.

Modules

Using standard library modules (math, random, os, datetime) and the third-party tkintermapview kept the code organised. Each import serves a clear purpose.

Classes

- Hospital bundles each hospital's data AND behaviour (status_colour()), keeping related code together.
- App holds the interface and its state, providing a clear container for all GUI-related methods.

Reusable Code

- build_banner() is reused on all three pages, eliminating duplicated code and ensuring consistency.
- build_faq() builds each FAQ item with the same structure.
- get_visible_hospitals() is reused by every redraw (search, sort, live update).

Readability

Named constants (WHITE, INK, GREEN, AMBER, RED, SHORT=30, MEDIUM=90) and comments marking where classes/functions/methods start make the code easy to navigate. Each function has a clear purpose and a docstring/comment explaining it.

Identify one programming convention you followed.

Convention: Named Constants Instead of Magic Numbers/Strings

```
# Colours and settings (constants - set once, use everywhere)
WHITE = "#FFFFFF"   # window background
INK   = "#26313F"   # main dark text
FONT  = "Segoe UI"  # the font used everywhere
GREEN = "#00A152"   # short wait
AMBER = "#E08E00"   # medium wait
RED   = "#E03A2F"   # long wait

SHORT  = 30   # under this many minutes = green
MEDIUM = 90   # under this many minutes = amber; otherwise red
```

Why it improved the program: A colour or threshold is defined once and used everywhere. Changing it is a one-line edit, and nothing drifts out of step. The names document what the value means. For example, SHORT=30 tells anyone reading the code that a "short wait" is under 30 minutes. If the threshold ever needs to change (say, to 20 minutes for a shorter target), I change one line and the entire program updates.

Describe your testing process.

Unit Testing:

- get_visible_hospitals(): Tests filtering and sorting with various queries
- status_colour(): Tests boundaries (29, 30, 89, 90 minutes)
- distance_km(): Tests known coordinate pairs
- load_hospitals(): Tests with valid data, missing file, bad lines, non-number waits

Integration Testing:

- Multi-page navigation: Search → Map → Detail → Back to previous page
- Live timer: Verifies wait times change and list/pins update
- Suburb selection: Checks distance calculation and map centring
- Sorting: Confirms mutual exclusivity of sort toggles

User Testing:

- Manual click-through (test steps 1-11 from the testing table)
- Stakeholder feedback rounds (Teodor, Sheroy, Teacher)

Boundary Testing:

- Wait thresholds: Tested at 29, 30, 89, 90 minutes
- Search: Empty string, single character, full name, partial match
- Distance: User suburb at same coordinates as hospital (0 km)

Invalid Data Testing:

- Missing hospitals.txt file
- Line with only 6 fields (skipped)
- Non-number wait time (skipped)
- Non-number coordinates (skipped)

Choose One Bug That Was Difficult to Fix

Bug: Map Pins Showing the Wrong Colour

Symptoms: North Shore Hospital's map pin was showing amber, but that same hospital's detail page showed the wait as green. The two colours disagreed for one specific hospital, and I noticed the pins never seemed to change colour even when wait times were clearly going up and down in the list and detail views.

Possible causes: Before I knew what was causing it, I suspected several possibilities. Perhaps status_colour() was returning different values for the same hospital depending on where it was called from. Maybe the data was being stored in two different places and one copy wasn't being updated. Perhaps the pins were showing old values from when the program first started and weren't refreshing. I also considered that the map library might be caching the markers and not allowing colour changes after creation.

Investigation: I started by comparing the pin colour to the live wait on the detail page for the same hospital at the same moment. The detail page would correctly show the updated wait and colour, but the pin would remain whatever colour it had at launch. I watched this over several update cycles and confirmed the pattern: the list and detail page would refresh correctly every three seconds, but the pins stayed frozen in their original colours. This pointed clearly at the markers being created only once at program start and never being refreshed. The list was being redrawn on every update, but the map pins were not.

Final solution: The root cause was that place_map_markers() only ran at start-up, so the pins were created with the initial wait colours and then never touched again. I fixed this by storing the markers in a list called self.hospital_markers and modifying update_times() to delete all existing markers and redraw them from scratch every three seconds. The fix works because the markers are now recreated with the current wait colours on every update cycle, just like the list is redrawn. The reasoning is that both the list and the map are visualisations of the same underlying data - the current wait time stored on each Hospital object - and they must both refresh from that single source of truth. If one view updates while the other doesn't, they drift out of sync and the user loses trust.

What I learnt: This bug taught me that live data has to refresh EVERY view of it, not just one. A single source of truth - the current wait time on each Hospital object - must drive the list, the pins and the detail page. If multiple views of the same data can get out of sync, the program loses trustworthiness. I also learnt that deleting and recreating objects is a valid pattern for keeping things in sync, and that storing objects as instance variables gives me the control I need to manage them properly. Finally, I learnt that subtle bugs like this - where one part of the interface is correct and another is wrong - can be much harder to spot than obvious crashes, and they require systematic comparison across views to identify.

Describe a Situation Where Testing Changed Your Original Design

Situation: Two Sort Checkboxes Causing Ambiguous Ordering

Original design: In Version 2, I had two independent checkboxes - "sort by shortest wait" and "sort by nearest" - each with its own Boolean variable. They were designed to work independently, so the user could tick either one or both. The code used an if/elif structure which silently gave priority to the wait sort when both were ticked, but I hadn't made this behaviour visible or exclusive to the user.

What testing revealed: During systematic testing of the sort functionality, I ticked both checkboxes at the same time. The program didn't crash - the list always sorted by wait time because the if/elif order gave wait priority, so "nearest" was silently ignored. The real problem was usability: a user who ticked "nearest" saw it stay ticked but got a wait-sorted list, with no sign that nearest was being ignored. The user would have no way of knowing that their selection wasn't actually taking effect.

The evidence that convinced me: This was row 1.20 of my testing table. It passed (no crash), but its recorded result is the evidence - Expected: "one clear sort order," Actual: "the wait sort takes priority over nearest… nearest is ignored when both are ticked. Not a crash, but a UX ambiguity." The testing table made it impossible to ignore - I had clear evidence that the design was flawed even though it technically worked. I also had evidence of the fix in row 1.40, whose result literally reads "the two sorts can no longer both be on, removing the V2 ambiguity." This before and after proof confirmed that the change was necessary and effective.

The design change: I made the two sorts mutually exclusive in Version 3. When the user ticks "sort by shortest wait," it automatically unticks "sort by nearest," and vice versa. I implemented this with two methods - on_sort_wait() and on_sort_nearest() - that check whether the other toggle is ticked and disable it if so. This means only one sort is ever active at a time, so the user always knows exactly how the list is ordered. The list will either be sorted by wait time, or by distance, never both or neither. The user can switch between them freely, and the interface clearly shows which sort is currently active.

Why the change improved the outcome: The change removes ambiguity and gives the user confidence in the information they're seeing. When the list is sorted, the user knows exactly what the order means - either "these are the shortest waits first" or "these are the closest hospitals first." This is essential for fitness for purpose because the user's decision depends on trusting the information. A confused user who can't tell how the list is ordered is less likely to trust the app and more likely to make a poor decision. The mutually exclusive design also makes the interface cleaner and simpler to understand, which matters in an emergency context where cognitive load is already high.

What this taught me: This experience taught me that passing tests isn't the same as good design - a feature can work technically but still be confusing for users. It also taught me the value of systematic testing with specific test cases. Without that row in my testing table documenting the UX ambiguity, I might not have noticed the problem or might have dismissed it as minor. The testing table provided clear evidence that forced me to address the issue properly. I also learnt that designing for clarity sometimes means restricting choices, and that's okay if it makes the user experience better.

Describe one piece of stakeholder feedback.

Feedback: Teodor's Map Suggestion (Version 2 → Version 3)

Why the stakeholder suggested it: Teodor liked the distance calculation in Version 2 but wanted to "visualise how far they are from the hospitals." A number like "12.3 km" gives a measurement, but a map gives spatial awareness — you can see which direction to travel and which hospitals are clustered together.

Whether I agreed: Yes. Visualising distance is a natural extension of calculating it. A map makes the information more intuitive and accessible.

What changes I made: I added an interactive tkintermapview map with colour-coded pins for each hospital and a blue "you are here" pin for the user's suburb. Clicking a pin opens the detail page.

How those changes improved the outcome: Users can now see the spatial relationship between their location and all hospitals at a glance. The colour-coded pins make wait status visible on the map, and clicking a pin gives full details. This transforms the app from a list-based tool to a visual decision-making aid.

Describe one stakeholder suggestion that you did NOT implement.

Feedback: Teodor's Google Maps Navigation Suggestion

What was suggested: Teodor suggested integrating Google Maps with live GPS location tracking and turn-by-turn directions to the hospitals. He felt that for people in a real emergency, accurate maps that can track their location and give them directions would be extremely valuable.

Why the stakeholder suggested it: Teodor was absolutely right that turn-by-turn directions would be extremely useful for someone heading to the emergency department. In a stressful situation, having the app not only tell you which hospital has the shortest wait but also guide you there would be incredibly helpful. His suggestion came from a genuine user need - when you're injured or unwell, you don't want to be fumbling with a separate maps app while trying to get to the right hospital.

Whether I agreed: I completely agreed that this would be a valuable feature. In fact, it would make the app much more useful and would be a natural next step after the user has decided which hospital to go to. Getting to the hospital is just as important as knowing which one to choose.

Why I did NOT implement it:

Technical constraints: Integrating Google Maps with turn-by-turn directions requires a Google Cloud API key, which involves setting up billing and ongoing costs. The free tier is limited, and if the app were ever shared or used by multiple people, costs could quickly escalate. This is beyond what is reasonable for a school assessment project where I don't have access to paid API services.

Complexity for a prototype: Implementing full GPS navigation would add significant complexity to the code. I would need to handle location permissions, real-time position tracking, route calculation, and dynamic map updates with directions overlaid. For a prototype that is meant to

demonstrate the core concept of comparing wait times, this level of complexity would have taken time away from building and refining the main features that users actually need to make a decision about which ED to go to.

Privacy implications: GPS tracking would require collecting real personal location data, which falls under the Privacy Act 2020. I would need explicit user consent, a clear privacy policy explaining how the data is used and stored, and would need to ensure the data is not retained beyond what is necessary. For a prototype built for a school assessment, adding these privacy considerations would add significant complexity without a corresponding educational benefit. The current approach of having users select their suburb from a dropdown gives them distance information without collecting any location data that could identify them.

What I did instead: I added an interactive map with colour-coded pins and a blue "you are here" marker showing the user's chosen suburb, which visualises the distance and location of each hospital. This gives users the spatial awareness they need to understand where hospitals are relative to them, without the complexity and privacy implications of full GPS navigation. Users can then easily open a separate navigation app on their phone if they need turn-by-turn directions - and many dedicated navigation apps are better at this than a prototype ever could be.

What this demonstrates: This decision shows that I evaluate feedback critically rather than accepting it all. I recognised that Teodor's suggestion was valuable and had merit, but I made a judgement call that it was out of scope for this prototype given the technical, practical and privacy constraints. A good developer doesn't just implement every feature request - they consider the purpose of the application, the resources available, and the implications of adding new features before making a decision. In a future version with proper resources and privacy protections, full navigation integration would be a fantastic addition.

Evaluation of Strengths of Your Completed Program

Colour Status on Dots, Pins and the Detail Page:

I implemented a consistent colour-coding system across every part of the interface - the list view shows coloured dots next to each hospital, the map pins are coloured to match the wait status, and the detail page displays the wait time in the same colour. This is a strength because it means the user can understand the situation at a glance without having to read and process numbers. In an emergency or stressful situation, cognitive load matters - being able to see green and know "short wait" immediately is faster and more reliable than reading "45 minutes" and having to interpret what that means. The consistency across all views also means the user only has to learn the colour system once, and it applies everywhere. This improves fitness for purpose because the core goal is helping people make a quick, informed decision about where to go, and visual status indicators support that goal far better than text alone.

Map with Distance and Live Pins:

The interactive map is a strength because it transforms abstract distance numbers into a spatial understanding that is much more intuitive. A number like "12.3 km" tells the user a measurement, but the map shows them direction, which hospitals are clustered together, and gives a realistic sense of the journey. The live colour-coded pins mean the user can scan the map and immediately see which hospitals in their area are

quiet and which are busy. The blue "you are here" marker anchored to their chosen suburb gives them an immediate reference point. This improves fitness for purpose because the user's decision isn't just about wait time or distance in isolation - it's about both, and the map allows them to weigh these factors together visually. A user can see that Hospital A is green but far away, while Hospital B is amber but close, and make a balanced choice.

Multi-Page Layout:

The three-page design is a strength because it removes visual clutter and reduces cognitive load. In Version 2, everything was on one page - the search, the list, the details, the map button, the FAQs - and it felt cramped. In Version 3, each page has one clear purpose: the search page is for finding and comparing hospitals, the map page is for visualising location, and the detail page is for focusing on one hospital. This means the user isn't overwhelmed with information they don't need at any given moment. When they want to see details, the detail page gives them the full picture without distractions. This improves fitness for purpose because a stressed user needs clarity, not density. A clean, focused interface is easier to process when you're in a hurry or worried.

Live Times with Last Updated Timestamp:

The live simulation that updates every three seconds, combined with the "last updated" timestamp, is a strength because it makes the data feel current and trustworthy. Even though the waits are simulated in this prototype, the behaviour of the app - the times changing, the colours updating, the timestamp refreshing - signals to the user that they are looking at live information. The timestamp is particularly important because it acknowledges the limitations of the data; it tells the user "this is the information available right now" rather than pretending to be perfectly accurate. This improves fitness for purpose because trust is essential in a health-related app. If a user doesn't trust the information, they won't use it to make a decision, and the app fails its purpose.

Safety Banner and Call 111 Button:

The permanent safety banner stating "For information only - not medical advice" and the red "Call 111" button are strengths because they establish the ethical boundaries of the app. The banner is always visible and never scrolls away, so the user is constantly reminded that this is a checking tool, not a diagnostic tool. The Call 111 button is prominent and red - the loudest element on the screen - so in a genuine emergency, the user is directed to the right place. This improves fitness for purpose because the app is designed to help with non-life-threatening urgent care decisions. By clearly separating itself from emergency services, it prevents misuse and protects the user from relying on it inappropriately. A health app that doesn't acknowledge its limitations is dangerous; this one does.

Robust File Loading:

The file loader that handles missing files, bad lines, and invalid data gracefully is a strength because it makes the app reliable. When hospitals.txt is missing, the app opens with an empty list and a message in the console rather than crashing. When a line in the data file is incomplete or has a non-number wait time, it's skipped and the rest of the data loads. This means the app always works, even if the data has minor issues. This

improves fitness for purpose because reliability is essential - if the app crashed when a user needed it, they would lose trust and abandon it. A robust app that always opens and functions, even with flawed data, is more dependable than one that breaks on the first error.

Mutually Exclusive Sorts:

The two sort options - "sort by shortest wait" and "sort by nearest" - being mutually exclusive is a strength because it removes ambiguity. In Version 2, both could be ticked at once, but the wait sort silently took priority while nearest was ignored, leaving the user confused. In Version 3, ticking one automatically unticks the other, so the user always knows exactly how the list is sorted. This improves fitness for purpose because a confused user is unlikely to trust the information they're seeing. When making a decision about where to go, clarity matters, and removing ambiguity from the interface supports better decision-making.

Search Functionality:

The case-insensitive substring search is a strength because it's forgiving and flexible. The user doesn't need to type the full hospital name or match capitalisation correctly - typing "north" will find North Shore Hospital, typing "shore" will also find it because the search checks for the substring anywhere in the name. This is especially important in an emergency context where the user might be stressed, typing quickly, or not thinking clearly. This improves fitness for purpose because the app is designed to be used by ordinary people in potentially stressful situations, and the search accommodates that reality rather than demanding precision.

Dedicated Detail Page:

The dedicated detail page with a large, colour-coded "hero" wait time is a strength because the most important information - how long the user will wait - is immediately visible and impossible to miss. The wait time is displayed in a large font and coloured green, amber or red so the user can understand the situation without reading. The page also shows the number of patients waiting, the phone number, and the hospital type in a clean, readable layout. This improves fitness for purpose because once the user has identified a potential hospital, they need clear, complete information to make their final decision. The detail page gives them that information without clutter or distraction.

What Evidence Shows That Your Program Is Fit for Purpose?

Testing Evidence

The comprehensive testing I carried out provides strong evidence that the program is fit for purpose. My headless test suite, which runs without a GUI, exercises the core logic including the filter and sort functions, the status colour boundaries, the distance calculation, and the file loader with both valid and invalid data. All 27 tests passed on the final version, confirming that the underlying logic is correct and reliable. This matters because a program that works for a few cases but breaks on others is not fit for purpose - users need to trust that the information they're seeing is accurate regardless of their specific circumstances.

Integration testing was equally important. I tested the multi-page navigation to ensure that switching between search, map and detail pages works smoothly and that the back button returns to the correct previous page. I verified that the live timer updates wait times correctly and that both the list and the map pins refresh to reflect those changes. I confirmed that selecting a suburb calculates the correct distances and updates both the list and the map. These tests prove that all the parts of the program work together as a whole, not just in isolation. A program that works in pieces but breaks when everything is connected is useless in practice.

Boundary testing provides evidence of robustness. I tested the colour thresholds at 29, 30, 89 and 90 minutes to confirm the green-amber-red transition works correctly at the exact boundaries. I tested the search with empty strings, single characters, full names and partial matches to ensure the substring filter works reliably. This proves the program handles edge cases properly, which is essential for fitness for purpose because users will inevitably test the boundaries whether the developer expects it or not.

Invalid data testing is perhaps the most important evidence of robustness. I deliberately removed hospitals.txt to confirm the app opens with an empty list rather than crashing. I created a line with only six fields instead of seven and verified it was skipped while the valid data loaded. I inserted a non-number wait time and confirmed that line was also skipped. This proves the program is resilient - even if the data file is corrupted or incomplete, the app still works. For a health-related tool, this reliability is non-negotiable.

Stakeholder Feedback Evidence

The stakeholder feedback from each version provides direct evidence that the program meets user needs. In Version 1, Teodor wanted distance information - I added the Haversine formula and distance display in Version 2. Sheroy wanted a 111 button and patient counts - I added the safety banner with the Call 111 dialog and the patient waiting count in the details panel. In Version 2, both Teodor and Sheroy wanted a map to visualise distance - I added the interactive map with colour-coded pins in Version 3. My teacher wanted the font size increased - I increased font sizes across the app in Version 3. The mutual-exclusion change for the sorts came from my own testing when I discovered that ticking both checkboxes in Version 2 silently ignored "nearest" without any indication to the user.

This matters because feedback is evidence that the program is fit for purpose - if the people who would actually use the app are asking for features and those features get built, it proves the program is moving in the right direction. The fact that each round of feedback was

implemented in the next version shows that the program evolved in response to real user needs, not just what I assumed would be useful. The iterative development process, driven by stakeholder input, produced an outcome that better serves its intended users.

Final Functionality Evidence

The final program does everything the brief requires and more. It compares Auckland Emergency Departments by wait time, displays each one with a green, amber or red colour for quick reading, allows searching by name, calculates and shows distance from the user's suburb, sorts by either wait time or distance, provides an interactive map with colour-coded pins, shows patient waiting counts and phone numbers, includes a safety banner and 111 button, and updates wait times live every three seconds with a timestamp.

All of these features work together to solve the original problem: there is currently no public place to compare ED wait times in New Zealand. Emergency Q fills that gap by putting all the information on one screen with clear visual indicators. A user can open the app, type their suburb, scan the list or map for a green hospital, check the distance, and make a decision - all in a matter of seconds. This is exactly what the purpose required.

The program also goes beyond the minimum requirements by being robust, visually polished, and professionally structured. The separation of logic from the GUI, the use of classes and objects, the file-based data loading, the live updates, the multi-page navigation, and the interactive map all demonstrate a level of quality that makes the program genuinely useful rather than just a proof of concept. The code is well commented, follows naming conventions, and uses named constants throughout, making it maintainable and adaptable for future development.

Overall Evidence of Fitness for Purpose

Taken together, the testing, the stakeholder feedback and the final functionality provide compelling evidence that Emergency Q is fit for purpose. The testing proves it works correctly, the feedback proves it meets user needs, and the final functionality proves it solves the original problem. A program that passes comprehensive tests, incorporates genuine user feedback, and delivers on its stated purpose is by definition fit for that purpose. The iterative development approach - building, testing, getting feedback, and improving - ensured that the final outcome was not just what I thought would work, but what actually works for real users in a real context.

References:

https://www.pythontutorial.net/tkinter/tkinter-stringvar/

https://www.pythontutorial.net/tkinter/tkinter-entry/

https://www.geeksforgeeks.org/python/python-os-path-abspath-method-with-example/

https://gist.github.com/nigelbrady/c3cda78dbc2019bd40d0

https://pypi.org/project/haversine/

https://www.movable-type.co.uk/scripts/latlong.html

https://docs.python.org/3/faq/programming.html

https://abd-01.github.io/posts/2024-07-22-python-latebind/

https://docs.python.org/3/howto/sorting.html

https://docs.python.org/3/library/__main__.html

https://peps.python.org/pep-0008/

https://docs.python.org/3/library/tkinter.html

https://tkdocs.com/shipman/universal.html

https://tkdocs.com/tutorial/

https://ccia.ugr.es/mgsilvente/tkinterbook/pack.htm

https://python-course.eu/tkinter/layout-management-in-tkinter.php

https://docs.python.org/3/tutorial/inputoutput.html

https://docs.python.org/3/tutorial/errors.html

https://docs.python.org/3/reference/import.html

https://docs.python.org/3/library/os.path.html

https://tkdocs.com/pyref/booleanvar.html